
task-app 30日間カリキュラム

Day 29

ユーザー詳細・編集ページを作ろう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	4
今日のゴール	4
始める前の前提	5
なぜこれを作るのか	6
実装ステップ一覧	10
Step 0: user.ts に getById / update を追記する(読む目安: 16分)	13
Step 1: 動的ルーティングの仕組みを理解する(読む目安: 5分)	25
Step 2: ユーザー詳細ページのファイルを作成する(読む目安: 5分)	27
Step 3: URLからユーザー ID を取得してデータを取得する(読む目安: 7分)	33
Step 4: グリッドレイアウトで詳細情報を表示する(読む目安: 7分)	42
Step 5: プロジェクト一覧とタスクテーブルを表示する(読む目安: 7分)	54
Step 6: 権限チェックで編集ボタンを出し分ける(読む目安: 5分)	64
Step 7: 編集ページのファイルを作成する(読む目安: 5分)	66
Step 8: zodスキーマと useForm でデータを同期する(読む目安: 7分)	84
Step 9: ロール選択・アクティブ状態の切り替えを実装する(読む目安: 7分)	90
Step 10: 保存機能を実装して完成(読む目安: 5分)	98
完成コード全体	114
今日のまとめ	166
つまずきポイント	166
今日学んだ用語	169
全体のデータフロー振り返り	169
理解チェック	173
追加課題: 別のユーザーを開いても前の編集値を残さない	174
次回予告	175
Day 29 完成形コード(参照用)	175

次に読むもの.....	175
奥付	176

前回の振り返り

Day 28 では **タスク一括操作**を実装しました。チェックボックスで複数タスクを選択し、一括でステータス変更・削除できる機能を作りました。今日は Day 24 で作った **ユーザー一覧ページ**の続きとして「各ユーザーをクリックしたときに開く詳細ページ」と「編集ページ」を作ります。

今日のゴール

ユーザーの詳細情報を表示するページと、管理者または本人がユーザー情報を編集できるページを作ります。Next.js の**動的ルーティング**という仕組みを使ってURLに含まれるユーザーIDからデータを取得する方法を学びます。

この日はまずサーバー側の `getById` と `update` を自分で書きます。そのあと2画面をつなぎます。

スクリーンショット: ユーザー詳細ページの完成イメージの表示を確認してください。

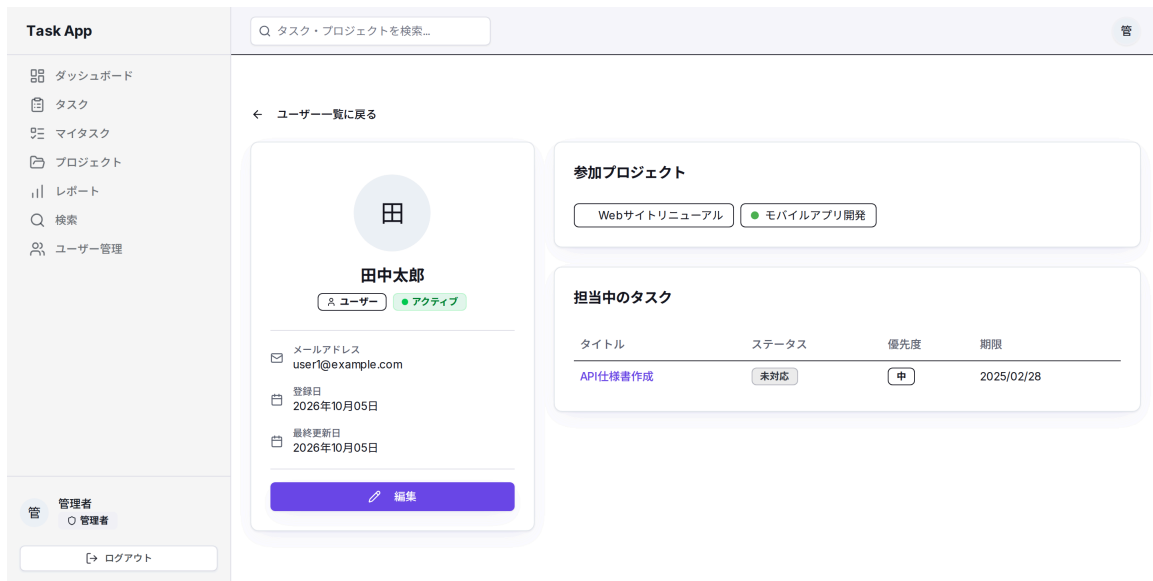


図1: ユーザー詳細ページ。左にアバターと名前とロールのバッジ、右に参加プロジェクトと担当中のタスクのカードが並んでいる

今日のゴールライン: 動的ルーティングでユーザー ID を受け取り、詳細表示から編集保存まで権限つきで動かせれば大丈夫です。

始める前の前提

- Day 24 のユーザー一覧ページが表示できます(詳細ボタンの遷移先は今日つくるため押すと404になります)
- 管理者ユーザーと一般ユーザーの両方で確認できます
- 編集対象にする練習用ユーザーが1人以上います
- URL の `[id]` 部分は実際のユーザー ID に置き換えて確認します

なぜこれを作るのか

管理者は「各ユーザーの情報を確認したい」「権限を変えたい」「アカウントを無効にしたい」という管理業務が必要です。また、一般ユーザーも自分のプロフィールを確認・編集する場面があります。

このアプリでは管理者は全ユーザーの情報を閲覧・編集でき、一般ユーザーは自分の情報だけを閲覧・編集できるという権限モデルです。

例え話: 会社の社員名簿を想像してください。人事担当者(管理者)は全社員のページを開けますが一般社員は自分のページだけです。

「部署変更」「在籍状態の変更」は人事担当者だけが行えます。今日はその仕組みを作ります。

ページ構成とデータの流れ

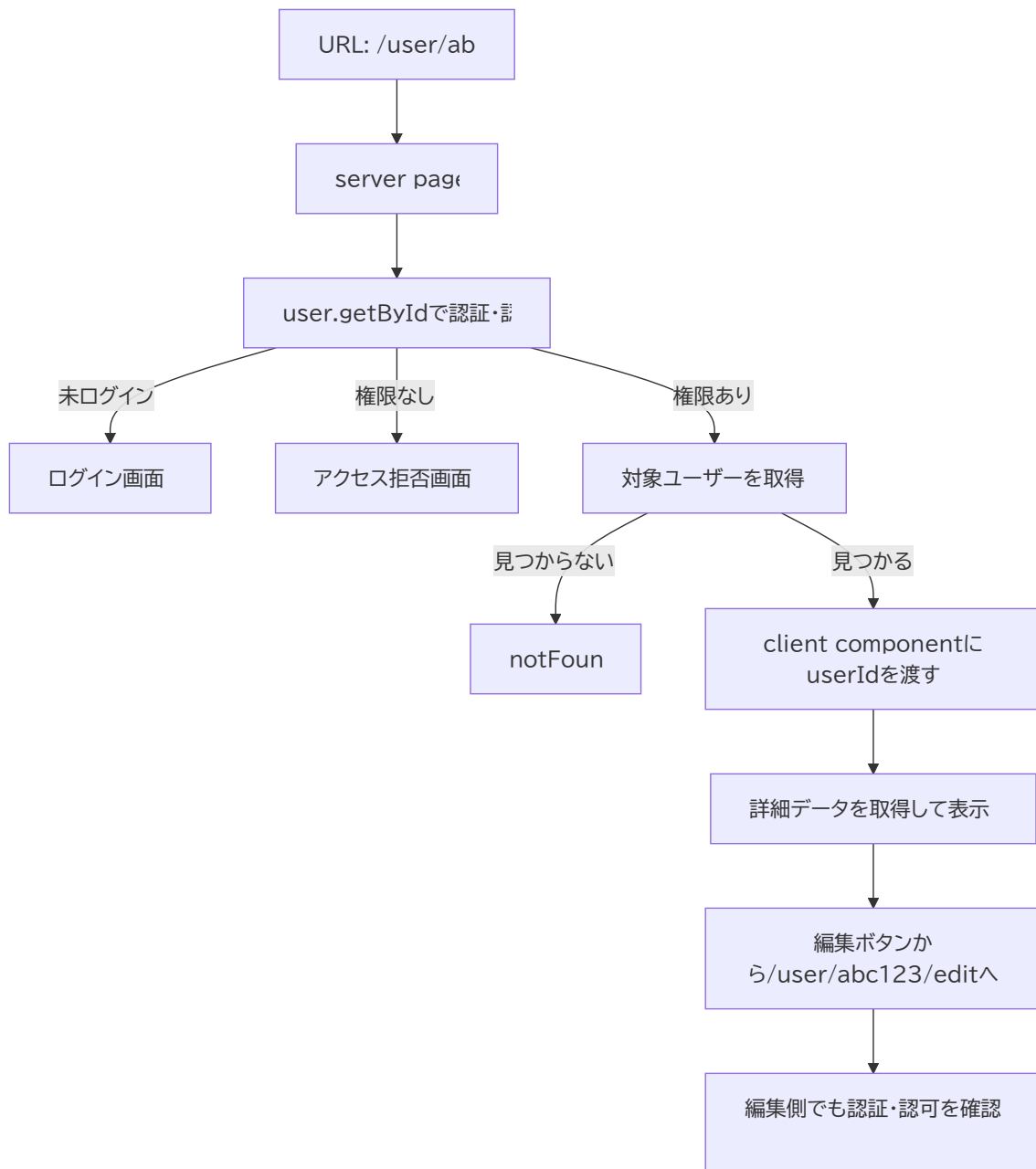


図2: ページ構成とデータの流れ

`page.tsx` は先に `user.getId` を呼び、ログイン状態と閲覧権限を確認します。この手続きは、本人と管理者のどちらにも当てはまらなければ対象の存在を調べる前に拒否します。他人のIDが存在するかどうかで応答を変えないためです。権限

Day 29: ユーザー詳細・編集ページを作ろう

がある場合だけユーザーを取得し、見つからなければ `notFound()` で404を表示します。取得できた場合はブラウザ側の部品へIDを渡します。ブラウザからの再取得にも同じ認可が適用されます。

やること / やらないこと

やること	やらないこと
動的ルーティング <code>[id]</code> フォルダの作成	パスワード変更機能
ユーザー詳細ページの実装	プロフィール画像のアップロード機能
管理者・本人向けユーザー編集ページの実装	メール通知機能
権限に基づいたUI表示切り替え	ユーザー削除機能
フォームとサーバーデータの同期 (<code>useEffect</code>)	2要素認証

新しく学ぶ概念

概念・読み方	役割	例え
動的ルーティング <code>[id]</code> 読み方: どうてきルーティング	URLのID部分を変数として受け取る	「社員番号001の名簿ページ」→ URLの001が変数
<code>notFound()</code> 読み方: ノットファウンド	そのIDが存在しないときに404へ送る	名簿にいない社員番号なら案内終了

概念・読み方	役割	例え
server wrapper 読み方: —	存在確認や404判定を server 側に寄せる	受付で本人確認してから 会議室へ通す
<code>await params</code> 読み方: アウエイト パラムズ	URLのパラメータ(変数) を server 側で受け取る	受付で渡された整理番号 を開いて読む
<code>useEffect</code> 読み方: ユーズエフェクト	取得したユーザー情報を フォームの値へ反映する ために使うフック	荷物が届いたら自動で棚 に並べる係
useForm + zod(復習) 読み方: —	フォーム管理+バリデー ション(Day 14 参照)	記入用紙のルール自動 チェック
権限チェック 読み方: けんげんチェック	ユーザーの役割によって 表示を変える	社員証の種類によって入 れる部屋を変える

開発サーバーは前の Day から動かしたまま使います。止めてあるときは

`npm run dev` で起動してから `http://localhost:3000` を開きます。

実装ステップ一覧

ステップ	作業内容・触るファイル・成功状態	読む時間の 目安
Step 0	作業内容: user.ts に getById / update を追記する 触るファイル: <code>src/server/api/routers/user.ts</code> 成功状態: getByIdとupdateを追加し、型検査が通る	16分
Step 1	作業内容: 動的ルーティングの仕組みを理解する 触るファイル: 概念説明のみ 成功状態: URLと[id]フォルダの対応を説明できる	5分
Step 2	作業内容: ユーザー詳細ページのファイルを作成 触るファイル: <code>src/app/user/[id]/page.tsx</code> 成功状態: server wrapper のファイルが存在する	5分
Step 3	作業内容: URLからユーザーIDを取得してデータを取得 触るファイル: <code>src/app/user/[id]/user-detail-client.tsx</code> 成功状態: ユーザー名が表示される	7分

ステップ	作業内容・触るファイル・成功状態	読む時間の 目安
Step 4	作業内容: グリッドレイアウトで詳細情報を表示 触るファイル: <code>src/app/user/[id]/user-detail-client.tsx</code> 成功状態: 2 カラムレイアウトで表示	7 分
Step 5	作業内容: プロジェクト一覧とタスクテーブルを表示 触るファイル: <code>src/app/user/[id]/user-detail-client.tsx</code> 成功状態: プロジェクトとタスクのリンクが表示される	7 分
Step 6	作業内容: 権限チェックで編集ボタンを出し分ける 触るファイル: <code>src/app/user/[id]/user-detail-client.tsx</code> 成功状態: 管理者・本人のみ編集ボタンが見える	5 分
Step 7	作業内容: 編集ページのファイルを作成 触るファイル: <code>src/app/user/[id]/edit/page.tsx</code> と <code>edit/user-edit-client.tsx</code> 成功状態: 2 ファイルが存在する	5 分
Step 8	作業内容: zod スキーマと useForm でデータを同期する 触るファイル: <code>src/app/user/[id]/edit/user-edit-client.tsx</code> 成功状態: 入力欄と form.register の記述が揃う	7 分

ステップ	作業内容・触るファイル・成功状態	読む時間の 目安
Step 9	作業内容: ロール選択・アクティブ状態の切り替え 触るファイル:	
	src/app/user/[id]/edit/user-edit-client.tsx 成功状態: SelectとCheckboxの値・変更処理が書けている	7分
Step 10	作業内容: 保存機能を実装して完成 触るファイル:	
	src/app/user/[id]/edit/user-edit-client.tsx 成功状態: 保存ボタンでDBが更新される	5分

表の Step 3 以降が `page.tsx` ではなく `user-detail-client.tsx` を指しているのは `page.tsx` を server wrapper(サーバー側で動く入口)にしてあるためです。この章の `page.tsx` は Server Component なので、画面の状態を扱う `useQuery` を直接呼べません。`useQuery` は、`'use client'` を付けた `user-detail-client.tsx` で呼びます。`page.tsx` というファイル名が理由ではありません。`page.tsx` の先頭に `'use client'` を書いて Client Component にすれば、`page.tsx` でも `useQuery` を呼べます。

読む時間の合計(仮): 約76分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 0: user.ts に getById / update を追記する(読む目安: 16分)

ゴール: Day 24・Day 25 で作った `src/server/api/routers/user.ts` に、`getById` と `update` を追記します。今日は UI で動的ルーティングを学びますがその前に「ユーザー詳細を返す入口」と「編集内容を保存する出口」を完成させます。

この2本で初めて Day 24 の一覧 → Day 29 の詳細 → Day 29 の編集、という流れが閉じます。ここも 完成版と同じく、**閲覧権限** と **更新権限** を先に判定してから DB を触ります。

最初に import 群を完成形へ置き換えます。`getById` の未完了タスク絞り込みに使う `TASK_STATUS` が今日の追加分です。

`Prisma` はDBのエラー型を実行中に判定するため、通常の `import` で読み込みます。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts(import 群の完成形)
import { Prisma } from '@prisma/client';
import { TRPCError } from '@trpc/server';
import bcrypt from 'bcryptjs';
import { z } from 'zod';
import { USER_ROLE } from '@lib/constant/roles';
import { TASK_STATUS } from '@lib/constant/status';
import { createPasswordSchema } from '@lib/password';
import { prisma } from '@lib/prisma';
import { createSession, type SessionUser } from
'@lib/session';
import { adminProcedure, createTRPCRouter,
protectedProcedure } from '../trpc';
import { USER_DETAIL_SELECT } from '../helpers/select';
import { isUserEmailUniqueConstraintError } from
'../helpers/user-email-conflict';
```

Day 25 で書いた `profileUpdateSchema` の前へ、管理者または本人が使う更新入力を追加します。

```
// filepath: src/server/api/routers/user.ts(profileUpdateSchema の前に追加)

const userUpdateSchema = z

  .object({

    id: z.string().cuid(),

    name: z.string().min(1,
'名前を入力してください').optional(),

    avatar: z.string().url().optional().nullable(),

    role: z.nativeEnum(USER_ROLE).optional(),

    isActive: z.boolean().optional(),

  })

  .refine(

    ({ name, avatar, role, isActive }) =>

      name !== undefined ||

      avatar !== undefined ||

      role !== undefined ||

      isActive !== undefined,

    '更新する項目を1つ以上指定してください',

  );
```

末尾の `.refine` は項目ごとの検査では捕まえられない条件を足すための書き方です。この `userUpdateSchema` は `id` 以外の4項目すべてに `.optional()` が付いているので `{ id }` だけを送っても項目単位の検査は通ります。それを許すと「何も変えない更新」が DB まで届き、`updatedAt` だけが動いたレコードが残ります。`.refine` で「4つのうち1つは入っていること」を最後に確かめるとその空振りの更新を入口で止められます。

0-1. getAll の直後に getById を足す

完成版のコードでは `getAll` の次が `getById` です。まずそこへ追記します。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts(getAll の直後に追加)

getById: protectedProcedure

  .input(z.object({ id: z.string().cuid() }))

  .query(async ({ ctx, input }) => {

// 本人またはADMINのみ他ユーザーの詳細情報にアクセス可能

    if (ctx.session.userId !== input.id &&
ctx.session.role !== USER_ROLE.ADMIN) {

      throw new TRPCError({

        code: 'FORBIDDEN',

        message: 'この操作を行う権限がありません',

      });

    }

  })
```

`getById` は管理者専用ではありません。本人なら自分の詳細を見られる必要があるからです。だから `protectedProcedure` を使い、`本人または ADMIN` という条件を中で判定しています。

```
// filepath: src/server/api/routers/user.ts(続き)
const user = await prisma.user.findUnique({
  where: { id: input.id },
  select: {
    ...USER_DETAIL_SELECT,
    createdAt: true,
    updatedAt: true,
    projects: {
      include: {
        project: {
          select: {
            id: true,
            name: true,
            color: true,
          },
        },
      },
    },
  },
},
```

ここからが「詳細ページに必要なデータ」を返す本体です。`USER_DETAIL_SELECT`に加えて登録日・更新日・所属プロジェクトを返します。

`projects.include.project.select` が少し深いですがこれは `ProjectMember` 経由でぶら下がっているプロジェクト本体の `id`・`name`・`color` まで一緒に返すためです。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts(続き)

    assignedTasks: {
      select: {
        id: true,
        title: true,
        status: true,
        priority: true,
        dueDate: true,
      },
      where: {
        status: {
          notIn: [TASK_STATUS.DONE,
TASK_STATUS.CANCELLED],
        },
        project: {
          members: {
            some: { userId: ctx.session.userId },
          },
        },
      },
      orderBy: { dueDate: 'asc' },
    },
  },
});
```

`assignedTasks` は「今その人が担当している作業」の一覧です。完了済みやキャンセル済みまで混ぜると詳細ページが読みづらくなるので

`notIn: [TASK_STATUS.DONE, TASK_STATUS.CANCELLED]` で除外します。

さらに、閲覧者が現在所属しているプロジェクトだけに絞ります。担当者本人や管理者であっても、現在所属していないプロジェクトのタスクをこの画面から読めないようにするためです。

```
// filepath: src/server/api/routers/user.ts(続き)

    if (!user) {
      throw new TRPCError({
        code: 'NOT_FOUND',
        message: 'ユーザーが見つかりません',
      });
    }

    return user;
  }),
```

`findUnique` は「無ければ null」を返すのでそのまま返さず `NOT_FOUND` に変換します。UI 側はこのエラーを受けて 404 やエラー-toast に繋がられます。

0-2. 次に update を追加する

`getById` の直後に `update` を追記します。この日の Step 0 で `profileUpdateSchema` の前に置いた `userUpdateSchema` をここで使います。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts(getById の直後に追加)

update: protectedProcedure.input(
  userUpdateSchema).mutation(async ({ ctx, input }) => {
  const { id, ...data } = input;

  if (id !== ctx.session.userId) {

// 他ユーザーを更新する場合はセッションのroleでADMIN判定

    if (ctx.session.role !== USER_ROLE.ADMIN) {
      throw new TRPCError({
        code: 'FORBIDDEN',
        message: '管理者権限が必要です',
      });
    }
  }
}
```

ここでもまず権限判定です。更新対象が自分以外なら管理者だけに許可します。判定に使っているのは `ctx.session.role` で、クライアントから送られてきた値ではありません。ここを `input` の中身で判定すると送信内容を書き換えるだけで誰でも管理者を名乗れてしまいます。0-1 の `getById` と判定材料をそろえてあるので閲覧と更新で権限の基準がずれません。

```
// filepath: src/server/api/routers/user.ts(続き)
    } else {

// 自分のプロフィール更新の場合、roleとisActiveは変更不可
        if (data.role !== undefined || data.isActive !==
undefined) {
            throw new TRPCError({
                code: 'FORBIDDEN',
                message:
'ロールとアクティブ状態は変更できません',
            });
        }
    }
}
```

本人編集のときは逆に制限が入ります。自分の名前やアバターは変えられても自分で `role` や `isActive` を変えるのは不可です。ここを入れておかないと一般ユーザーが DevTools から管理者権限を送り込めてしまいます。

```
// filepath: src/server/api/routers/user.ts(続き)
const updateData: Prisma.UserUpdateInput = {};
if (data.name !== undefined) {
  updateData.name = data.name;
}
if (data.avatar !== undefined) {
  updateData.avatar = data.avatar;
}
if (data.role !== undefined) {
  updateData.role = data.role;
}
if (data.isActive !== undefined) {
  updateData.isActive = data.isActive;
  updateData.sessionVersion = { increment: 1 };
}
```

ここは「渡された項目だけを更新対象に足す」段階です。空文字や `undefined` を無理にまとめて送らず、存在する項目だけ 1 本ずつ加えています。管理者が `isActive` を更新したときは、アカウント無効化・再有効化に伴い `sessionVersion` を `increment: 1` します。無効化前に取得された古いトークンが再有効化時に復活して悪用されるのを防ぐためです。

```
// filepath: src/server/api/routers/user.ts(続き)
try {
  return await prisma.user.update({
    where: {
      id,
      ...(data.isActive !== undefined
        ? { sessionVersion: { lt:
POSTGRES_INTEGER_MAX } }
        : {}),
    },
    data: updateData,
    select: {
      ...USER_DETAIL_SELECT,
      updatedAt: true,
    },
  });
}
```

`isActive` を更新する際は

`sessionVersion: { lt: POSTGRES_INTEGER_MAX }` の上限チェックを行います。数値あふれを防ぐための安全策です。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts(続き)
    } catch (err) {
      if (err instanceof
Prisma.PrismaClientKnownRequestError && err.code ===
'P2025') {
        if (data.isActive !== undefined) {
          throw new TRPCError({
            code: 'CONFLICT',
            message:
'セッションを更新できません。管理者にお問い合わせください',
          });
        }
        throw new TRPCError({
          code: 'NOT_FOUND',
          message: 'ユーザーが見つかりません',
        });
      }
      throw err;
    }
  }
},
```

更新後は画面がすぐに描画し直せるよう `USER_DETAIL_SELECT` と `updatedAt` を返します。更新時点で対象が存在しない場合、DBは `P2025` を返します。

`isActive` 変更時の `P2025` は版数あふれの可能性があるため `CONFLICT` を返し、通常の未発見時は `NOT_FOUND` に変換します。その他のエラーは投げ直します。ここまでで `update` は完成です。

0-3. 追記後の user.ts 全体を確認する

今日ここまで追記すると `src/server/api/routers/user.ts` には次の5本が揃います。

順番	procedure	追加した日
1	<code>getAll</code>	Day 24
2	<code>getById</code>	Day 29
3	<code>update</code>	Day 29
4	<code>updateProfile</code>	Day 25
5	<code>changePassword</code>	Day 25

順番も 完成版と同じです。`root.ts` の `user: userRouter` は Day 24 です
で登録済みなので今日も root の追記はありません。

確認ポイント:

- `src/server/api/routers/user.ts` に `getById` と `update` を 完成版と同じ位置へ追記できました
- `getById` が「本人または ADMIN」、`update` が「本人更新 / 他人更新」で分岐しています
- Day 24～29 の積み上がりで `userRouter` の5手続きが揃いました
- `npm run tsc --noEmit` で型エラーが出ていません

Step 1: 動的ルーティングの仕組みを理解する(読む目安: 5分)

ゴール: URLのどの部分を `[id]` フォルダが受け取るか説明できるようになります。

Day 29: ユーザー詳細・編集ページを作ろう

Next.js ではフォルダ名を `[id]` のように角括弧で囲むと、そのフォルダ名が「変数」になります。URL の対応する部分が自動的にその変数に入ります。

フォルダ構造:

```
src/app/user/[id]/page.tsx
```

アクセスできるURL:

```
/user/abc123    → id = "abc123"  
/user/xyz789    → id = "xyz789"  
/user/user001   → id = "user001"
```

これが動的ルーティングです。1つのファイルで何千人ものユーザーページを作れます。

方式	フォルダ例	動作
静的ルーティング	<code>src/app/about/page.tsx</code>	<code>/about</code> だけに対応
動的ルーティング	<code>src/app/user/[id]/page.tsx</code>	<code>/user/なんでも</code> に対応

`[id]` の `id` という名前は自由に決められます。`[userId]` でも `[username]` でも OK です。ただしコード内で読み取るときも同じ名前を使います。

次のコードはURLとファイルの対応を説明する例です。ここでは入力せず、ファイルは Step 2 で作成します。

```
// filepath: src/app/user/[id]/page.tsx
// このファイルは /user/なんでも というすべてのURLに対応する
// URLの「なんでも」部分が params['id'] として受け取れる
export default function UserDetailPage() {

  // 次のステップでここに params から id を取り出す処理を書く
  return <div>ユーザー詳細ページ</div>;
}
```

この例は `params` を受け取らず、どのIDのURLでも「ユーザー詳細ページ」という文字を返します。実際にユーザーごとの情報を表示するにはURLのIDを読み取る処理も必要です。Step 2 でその処理を書きます。

確認ポイント:

- 動的ルーティングは「角括弧 `[]` でフォルダ名を囲む」ことで実現します
- 1つのファイルで無数のURLに対応できる仕組みだと理解しました

Step 2: ユーザー詳細ページのファイルを作成する(読む目安: 5分)

ゴール: 必要なフォルダとファイルを作成し、まず骨組みを作ります。

以下のフォルダ構造を作成します。

Day 29: ユーザー詳細・編集ページを作ろう

```
src/app/user/  
├─ page.tsx      ← Day 24 で作った一覧ページ  
└─ [id]/  
    ├─ page.tsx  ← 今回作成(詳細ページ)  
    └─ edit/  
        └─ page.tsx ← 後で作成(編集ページ)
```

`[id]` の下にさらに `edit/` を置くのはURLの入れ子とフォルダの入れ子を同じ形にそろえるためです。`/user/abc123` は `[id]/page.tsx` が受け持ち、`/user/abc123/edit` は `[id]/edit/page.tsx` が受け持ちます。編集ページ側でも同じ `[id]` の値を読めるので詳細から編集へIDを持ち回す仕組みを別に作る必要はありません。

作成した `page.tsx` に、次のコードを書きます。この入口は認証・認可・404をブラウザへ渡す前に判定します。この教材では、この役目を持つ入口を `server wrapper` と呼びます。

```
// filepath: src/app/user/[id]/page.tsx
import { getTRPCErrorFromUnknown } from '@trpc/server';
import { notFound, redirect } from 'next/navigation';
import { AppLayout } from '@component/layout/app-layout';
import { Card, CardContent } from '@component/ui/card';
import { trpc } from '@trpc/server';
import { UserDetailsClient } from './user-detail-client';

interface UserDetailsPageProps {
  params: Promise<{
    id: string;
  }>;
}

export default async function UserDetailsPage({ params }:
UserDetailsPageProps) {
  const { id } = await params;

  try {
    await trpc.user.getById({ id });
  } catch (error) {
    const trpcError = getTRPCErrorFromUnknown(error);
```

Day 29: ユーザー詳細・編集ページを作ろう

`params` が `Promise` になっているのは Next.js 15 から動的ルーティングの値が非同期で渡されるためです。`await params` で中身を開いてから `id` を取り出します。`trpc` はスターターが `src/trpc/server.ts` に配る server 用の呼び出し口です。ブラウザ用の `api` と違い、Server Component から手続きを直接呼べます。

エラーの種類ごとに行き先を分けます。

```
// filepath: src/app/user/[id]/page.tsx(同じファイルの続き)
if (trpcError.code === 'UNAUTHORIZED')
  redirect('/login');
if (trpcError.code === 'NOT_FOUND') notFound();
if (trpcError.code === 'FORBIDDEN') {
  return (
    <AppLayout>
      <div className="container mx-auto max-w-6xl mt-8">
        <Card>
          <CardContent className="pt-6">
            <h1 className="text-2xl font-bold mb-2">
              アクセス権がありません
            </h1>
            <p className="text-muted-foreground">
              このユーザーを見る権限がありません
            </p>
          </CardContent>
        </Card>
      </div>
    </AppLayout>
  );
}
```

未ログインはログイン画面、権限がある人から見て対象が無い場合は404、権限がない場合は拒否画面です。`getById` は対象を検索する前に本人か管理者かを調べるので、権限のない人には実在するIDと存在しないIDのどちらでも同じ拒否画面が出ます。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/page.tsx(同じファイルの続き)
    throw error;
  }

  return <UserDetailClient userId={id} />;
}
```

想定外のエラーは `throw error` で `error.tsx` へ渡します。500相当の失敗まで404や権限拒否に置き換えると、障害と入力間違いを区別できません。手続きを通過した場合だけ `UserDetailClient` へIDを渡します。

ここでブラウザを開くのはまだ早いです。1行目で読み込んでいる

`./user-detail-client` を、次の Step 3 で作るためです。この時点で

`/user/...` を開くと画面いっぱいに

`Module not found: Can't resolve './user-detail-client'` と表示されます。これは書き間違いではなく、まだファイルが無いだけです。

動作確認は Step 3 でそのファイルを作ってから行います。そのときに使う実在のユーザーIDはDay 24のユーザー一覧、またはDBの `users` テーブルで確認できます。

確認ポイント:

- `src/app/user/[id]/page.tsx` ファイルが作成できました
- `src/app/user/[id]/page.tsx` が server wrapper になり、`trpc.user.getById` を先に呼んでいます
- 未ログイン・権限なし・未発見・想定外エラーの4つを別々に扱っています
- この時点ではブラウザで開かず、次の Step へ進みます

Step 3: URLからユーザーIDを取得してデータを取得する(読む目安: 7分)

ゴール: `user-detail-client.tsx` でユーザーを取得し、読み込み・権限拒否・未発見・通信失敗を画面で区別します。

まず、この Step で使う部品を取り込みます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx
'use client';

import { useRouter } from 'next/navigation';
import { AppLayout } from '@component/layout/app-layout';
import { Button } from '@component/ui/button';
import { PageLoadingSpinner } from
  '@component/ui/loading-spinner';
import { USER_ROLE } from '@lib/constant/roles';
import {
  httpStatusOf,
  isAuthError,
  isForbiddenError,
  shouldRetryQuery,
} from '@lib/query-error';
import { api } from '@trpc/react';

const shouldRetryUserQuery = (failureCount: number,
error: unknown) =>
  httpStatusOf(error) !== 404 && shouldRetryQuery(
failureCount, error);
```

Day 29: ユーザー詳細・編集ページを作ろう

`shouldRetryQuery` は一時的な通信失敗だけを再試行します。401・403に加えて404も繰り返しても結果が変わらないため、ユーザー取得では再試行から外します。

2つの query からデータと状態を受け取ります。

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

interface UserDetailsClientProps {
  userId: string;
}

export function UserDetailsClient({ userId }:
UserDetailsClientProps) {
  const router = useRouter();

  const {
    data: currentUser,
    isLoading: isCurrentUserLoading,
    isError: isCurrentUserError,
    isFetching: isCurrentUserFetching,
    error: currentUserError,
    refetch: refetchCurrentUser,
  } = api.auth.getCurrentUser.useQuery(
    undefined,
    { retry: shouldRetryQuery },
  );
};
```

`isLoading` は初回取得中、`isFetching` は再取得中也含む状態です。再試行ボタンを連打できないようにするには後者も必要です。

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

const {
  data: user,
  isLoading: isUserLoading,
  isError: isUserError,
  isFetching: isUserFetching,
  error: userError,
  refetch: refetchUser,
} = api.user.getById.useQuery(
  { id: userId },
  {
    enabled: userId.length > 0,
    retry: shouldRetryUserQuery,
  },
);

const queryErrors = [
  isCurrentUserError ? currentUserError : null,
  isUserError ? userError : null,
];

const authFailed = queryErrors.some(isAuthError);
const forbidden = queryErrors.some(isForbiddenError);
const notFound = isUserError && httpStatusOf(userError)
=== 404;
```

queryErrors に2本の失敗を集めます。どちらかが401または403なら、前回のユーザーデータを隠すためです。ログアウト後もキャッシュが残ることはあります。その内容を拒否画面の裏に残してはいけません。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

const hasFetchError = isCurrentUserError || isUserError;

const hasRequiredData =
  (!isCurrentUserError || currentUser != null)
  && (!isUserError || user != null);

const requiredLoading = isCurrentUserLoading ||
isUserLoading;

const requiredFetching = isCurrentUserFetching ||
isUserFetching;

const refetchRequiredData = () => {
  void refetchCurrentUser();
  void refetchUser();
};

if (requiredLoading && !authFailed && !forbidden
&& !notFound) {
  return (
    <AppLayout>
      <PageLoadingSpinner />
    </AppLayout>
  );
}
```

読み込み中だけスピナーを表示します。取得済みデータがないまま失敗した場合は、次の分岐へ進んで理由と操作を表示します。これで通信が終わった後に待ち続ける状態を防ぎます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

if (
  authFailed
  || forbidden
  || notFound
  || (hasFetchError && !hasRequiredData)
  || !currentUser
  || !user
) {
  return (
    <AppLayout>
      <div className="flex flex-col items-center
justify-center py-24 text-center">
        <p className="mb-2 text-base font-semibold
text-foreground">
          {authFailed
            ? 'ログインの有効期限が切れました'
            : forbidden
            ? 'このユーザーを見る権限がありません'
            : notFound
            ? 'ユーザーが見つかりません'
            : 'ユーザー情報を取得できませんでした'}
        </p>
      </div>
    </AppLayout>
  )
}
```

最初の早期リターンで表示する見出しをエラー別に切り替えます。ログイン切れ、権限不足、未発見、通信失敗を同じ文言にすると、読者が次に何をすればよいか判断できないためです。

Day 29: ユーザー詳細・編集ページを作ろう

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き) */}

<p className="mb-6 text-sm text-muted-foreground">
  {authFailed
    ? 'もう一度ログインしてください。'
    : forbidden
    ? '管理者または本人のみ表示できます。'
    : notFound
    ?
    '削除されたか、URLが正しくない可能性があります。'
    :
    '通信状況を確認して、再読み込みしてください。'}
</p>

<Button
  type="button"
  onClick={() => {
    if (authFailed) {
      router.push('/login');
      return;
    }
    if (forbidden) {
      router.push('/dashboard');
      return;
    }
  }}
/>
```

案内文の下に、その状態から離れるためのボタンを置きます。401と403は取得を繰り返しても成功しないので、再試行ではなくログイン画面またはダッシュボードへ移動させます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

    if (notFound) {
      router.push(
        currentUser?.role === USER_ROLE.ADMIN
          ? '/user'
          : '/dashboard',
      );
      return;
    }

    refetchRequiredData();
  }}

  disabled={requiredFetching}
>
  {authFailed
    ? 'ログイン画面へ'
    : forbidden
    ? 'ダッシュボードへ'
    : notFound
    ? currentUser?.role === USER_ROLE.ADMIN
      ? 'ユーザー一覧へ'
      : 'ダッシュボードへ'
    : '再読み込み'}

</Button>
```

Day 29: ユーザー詳細・編集ページを作ろう

未発見の場合は、管理者ならユーザー一覧、一般ユーザーならダッシュボードへ戻します。その他の取得失敗だけを `refetchRequiredData` に渡し、同じ認可失敗を無意味に繰り返さない形にします。

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き) */}

    </div>
  </AppLayout>
);
}

const isAdmin = currentUser.role === USER_ROLE.ADMIN;
const isOwnProfile = currentUser.id === user.id;
```

401ではログイン、403ではダッシュボード、404では管理者だけユーザー一覧へ戻します。一時的な500相当の失敗で前回データがある場合はこの早期リターンを通らず、Step 4で追加する警告と前回内容を表示します。本人判定にはサーバーが返した `user.id` を使います。

確認ポイント: 存在しないIDにアクセスすると server wrapper から

`notFound()` が呼ばれ、`src/app/not-found.tsx` の404画面が表示されます。client側で再取得中に対象が消えた場合も、スピナーではなく未発見の案内が残ります。

正常系の表示を書きます。次のステップで内容を充実させます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx
return (
  <AppLayout>
    <div className="container mx-auto max-w-6xl py-8">
      <h1 className="text-2xl font-bold">{
user.name}</h1>
      <p>ID: {userId}</p>
    </div>
  </AppLayout>
);
}
```

名前とIDの2つしか出さないのはデータが本当に届いているかを最短で確かめるためです。レイアウトを先に作り込んでしまうと画面が空のときに原因を絞り込めません。データが来ていないのか、権限で弾かれたのか、CSS で見えなくなっているのかが混ざります。名前が出た時点で、URL から `getById` までの経路は通ったと確定します。飾りを足すのはそのあとです。

ここで初めてブラウザを開きます。Step 2 で読み込んでいたファイルがそろったので今度は `Module not found` が出ません。

開発サーバーが動いていればそのまま使います。止めてあるときだけ次のコマンドで起動します。

```
# filepath: ターミナル
# 停止している場合だけ起動する
npm run dev
```

Day 29: ユーザー詳細・編集ページを作ろう

実在するユーザーIDを Day 24 のユーザー一覧で確認し、`/user/そのID` を開きます。データが届くまでの一瞬は次のように本文の場所がスピナーだけになります。



図3: 読み込み中のユーザー詳細ページ。左のサイドバーは出たままで右の本文の場所に丸いスピナーだけが回っている

スピナーが消えたあとにユーザー名とIDが出ればURLから `getId` までの経路がつながっています。

確認ポイント:

- 存在するユーザーIDでアクセスするとユーザー名が表示されます
- 完成版では存在しないIDは Step 2 の `page.tsx` が404へ流します
- 読み込み中はスピナーが表示されます

Step 4: グリッドレイアウトで詳細情報を表示する(読む目安: 7分)

ゴール: 戻るボタン・サイドバー・メインコンテンツの2カラムレイアウトで表示します。

モバイル: 縦に並ぶ

PC: 左4列 + 右8列

Avatar	プロジェクト一覧		
名前	担当タスクテーブル		
ロール			

左を4列、右を8列に分けるのは左が名前やロールのような長さの決まった情報で、右が件数によって伸び縮みする一覧だからです。伸びる側に広い幅を渡します。スマートフォンの幅では12列を横に割ると1つが狭くなりすぎるので `md:` を付けて画面が広いときだけ2カラムにします。狭い画面では上下に積まれます。

必要なコンポーネントをインポートします。ファイルの先頭のインポート部分に追加してください。

```
// filepath: src/app/user/[id]/user-detail-client.tsx
import { format } from 'date-fns';
import { ja } from 'date-fns/locale';
import { ArrowLeft, Calendar, Mail } from 'lucide-react';
import { Avatar, AvatarFallback, AvatarImage }
  from '@component/ui/avatar';
import { Card, CardContent } from '@component/ui/card';
import { Separator } from '@component/ui/separator';
import { ActiveStatusBadge, UserRoleBadge }
  from '@component/ui/user-badges';
import { formatDateOnly } from '@lib/date';
```

Day 29: ユーザー詳細・編集ページを作ろう

日付の道具が2種類あることに気付いたでしょうか。`format` は時刻まで持っている値を日本語の表記に直すもので登録日と最終更新日に使います。

`formatDateOnly` は期限のように「日付だけ」の値に使います。期限は時刻を持たない約束で保存してあるため時刻として読み直すと表示が1日ずれます。混ぜて使わないよう、最初から両方を取り込んでおきます。`ActiveStatusBadge` と `UserRoleBadge` を自作せず取り込むのは一覧ページと同じ色と文言をそのまま使うためです。

12列グリッド(`md:grid-cols-12`)で左4列・右8列に分割します。`return` 文を書き換えます。まず、Step 3 で残した前回データを表示するときの警告を置きます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx

return (
  <AppLayout>
    <div className="container mx-auto max-w-6xl py-8">
      {hasFetchError && (
        <div
          role="alert"
          className="mb-4 flex items-center
justify-between gap-4 rounded-lg border
border-amber-300/60 bg-amber-50 px-4 py-3 text-sm
text-amber-900 dark:border-amber-500/40
dark:bg-amber-950/40 dark:text-amber-200"
        >
          <span>
最新のユーザー情報を取得できませんでした。前回取得時の内容です。</span>

          <Button type="button" variant="outline"
size="sm"
onClick={refetchRequiredData} disabled={
requiredFetching}>
再試行
</Button>
        </div>
      )}
    </div>
  )}

```

取得に失敗しても前回データが残っている場合だけ警告を出します。「再試行」は Step 3 の `refetchRequiredData` を使うので、現在のユーザーと表示対象のユーザーをまとめて取り直します。

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き) */}

<Button
  variant="ghost"
  className="mb-4 pl-0 hover:bg-transparent
hover:text-primary"
  onClick={() => router.push(isAdmin ? '/user' :
'/profile')}
>
  <ArrowLeft className="mr-2 h-4 w-4" />
  {isAdmin ? 'ユーザー一覧に戻る' :
'プロフィールに戻る'}
</Button>

<div className="grid gap-6 md:grid-cols-12">
```

戻るボタンを画面の先頭に置いているのはこの詳細ページがURLを直接開いても表示できるからです。一覧を経由せずに来た人はブラウザの戻るボタンで元の画面へ帰れません。管理者はユーザー一覧へ戻し、一般ユーザーは自分のプロフィールへ戻します。一般ユーザーには権限のない `/user` を行き先にしないため、`isAdmin` で行き先と文言を切り替えます。最後の `md:grid-cols-12` で12列の枠を開き、この中に左右のカラムを入れていきます。

左カラムにアバターとユーザー基本情報を置きます。

```

    { /* filepath:
src/app/user/[id]/user-detail-client.tsx */}
    { /* 左カラム: アバター・基本情報 */}
    <div className="md:col-span-4 space-y-6">
      <Card>
        <CardContent className="pt-6">
          <div className="text-center mb-6">
            { /* アバター画像(未設定時は名前の頭文字
を表示) */}

            <Avatar className="w-24 h-24 mx-auto
mb-4">

              {user.avatar && <AvatarImage
                src={user.avatar}
                alt={user.name || ''} />}
              <AvatarFallback className="text-3xl">
                {user.name?.[0]?.toUpperCase()}
              </AvatarFallback>
            </Avatar>
            <h2 className="text-xl font-bold mb-2">
              {user.name}
            </h2>
            { /* ロールバッジとアクティブ状態バッジ
*/}

            <div className="flex justify-center
gap-2 mb-4">

              <UserRoleBadge role={user.role} />

```

Day 29: ユーザー詳細・編集ページを作ろう

```
        <ActiveStatusBadge isActive={
user.isActive} />
      </div>
    </div>
```

`user.avatar` と `user.name` はどちらもデータベース側で空を許している列です。ですから両方とも「入っていない人がいる」前提で書きます。

`{user.avatar && <AvatarImage ... />}` はURL が無い人には画像そのものを置かないという意味です。囲まずに書くと行き先の無い画像を読み込もうとして枠が壊れます。名前側の `user.name?.[0]` も同じ理由です。`?.` を外すと名前未設定のユーザーのページは Day 26 で作った `error.tsx` のエラー画面に切り替わります。頭文字を出す `AvatarFallback` は画像が無い人の丸が空白にならないための受け皿です。

次にセパレーターとメールアドレスを表示します。

```

{/* filepath: src/app/user/[id]/user-detail-client.tsx */}
{/* セパレーターとメールアドレス表示 */}

    <Separator className="my-4" />
    <div className="space-y-4 text-sm">
      <div className="flex items-center
gap-3">

        <Mail className="h-4 w-4
text-muted-foreground" />

        <div>
          <p className="font-medium
text-muted-foreground text-xs">
            メールアドレス
          </p>
          <p>{user.email}</p>
        </div>
      </div>
    </div>

```

Separator を1本挟むのは上と下で情報の性質が変わるからです。上は名前とバッジで「この人が誰か」、下はメールと日付で「この人の属性」です。線が無いと縦に9行並ぶだけになり、目がどこで区切れればよいか迷います。ここから始まる3行はアイコン・小さい見出し・値、という同じ形の繰り返しです。最初の1つで形を決めておくと残りは同じ型に流し込むだけになります。

確認ポイント: メールアドレスがアイコン付きで表示されることを確認してください。

登録日も同じレイアウトパターンで表示します。

Day 29: ユーザー詳細・編集ページを作ろう

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx */}
{/* 登録日の表示 */}

    <div className="flex items-center
gap-3">

        <Calendar className="h-4 w-4
text-muted-foreground" />

        <div>

            <p className="font-medium
text-muted-foreground text-xs">
                登録日
            </p>

            <p>
                {user.createdAt
                    ? format(
                        new Date(user.createdAt),
                        'yyyy年MM月dd日',
                        { locale: ja }
                    )
                    : '-'}
            </p>
        </div>
    </div>
```

表示の形を `'yyyy年MM月dd日'` に決めて時刻を落としています。登録日を見る人が知りたいのは「いつからいる人か」で、何時何分かは判断に使わないからです。値が無いときに `-` を出しているのは空文字にすると「登録日」という見出しだけが値なしで残るためです。読む側はまだ読み込み中なのか、そもそも値が無いのか区別できません。`-` は「確かめたうえで空だった」という返事になります。

確認ポイント: 登録日がカレンダーアイコン付きで表示されることを確認してください。

最終更新日も同じパターンで表示します。

Day 29: ユーザー詳細・編集ページを作ろう

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx */}
{/* 最終更新日と左カラムの閉じタグ */}

    <div className="flex items-center
gap-3">

        <Calendar className="h-4 w-4
text-muted-foreground" />

        <div>

            <p className="font-medium
text-muted-foreground text-xs">
                最終更新日
            </p>

            <p>
                {user.updatedAt
                    ? format(
                        new Date(user.updatedAt),
                        'yyyy年MM月dd日',
                        { locale: ja }
                    )
                    : '-'}
            </p>
        </div>
    </div>
</CardContent>
</Card>
</div>
```

最終更新日を出しておくこと今日の後半で作る保存機能を自分で検証できます。データベース側の `updatedAt` はレコードが更新されるたびに自動で書き換わる列です。Step 10 で名前を保存して詳細ページに戻ったときこの日付が今日に変われば保存が本当に届いたと分かります。トーストが出ただけでは通知が出ただけかもしれません。画面に残る証拠を1つ持っておきます。

確認ポイント: メールアドレス・登録日・最終更新日がアイコン付きで表示されていることを確認してください。

左カラムの `div` を閉じた後、右カラムの枠を書きます。中身は Step 5 で追加します。

```
        {/* filepath:
src/app/user/[id]/user-detail-client.tsx */}
        {/* 右カラム: Step 5で中身を追加 */}
        <div className="md:col-span-8 space-y-6">
            {/* Step 5 でプロジェクト・タスクを追加 */}
        </div>
    </div>
</div>
</AppLayout>
);
}
```

Day 29: ユーザー詳細・編集ページを作ろう

中身が空のまま右カラムを置くのは **4** と **8** を足して12列になっているかを今のうちに確かめるためです。数が合っていないと右のカラムが左の下へ回り込みます。カードを入れてからその崩れに気付くと原因が列の数なのかカードの幅なのか切り分ける手間が増えます。今なら右側に空きスペースが見えるかどうかだけで判断できます。ブラウザの幅を狭めて縦積みにも変わることも、ここで一緒に確かめます。

確認ポイント:

- PCサイズのブラウザで左にアバター、右にスペースが表示されます
- スマホサイズに縮小すると縦に並びます
- アバターが未設定のユーザーでは名前の頭文字が表示されます
- 管理者には「ユーザー一覧に戻る」、一般ユーザーには「プロフィールに戻る」ボタンが表示されます

Step 5: プロジェクト一覧とタスクテーブルを表示する(読む目安: 7分)

ゴール: 参加プロジェクトと担当タスクを、キーボードでも開けるリンクとして表示します。

必要なコンポーネントをインポートします。Step 4 で追加した **Card** の import には **CardHeader** と **CardTitle** を足し、次の1行へ置き換えてください。その他の import はファイル先頭へ追加します。

```
// filepath: src/app/user/[id]/user-detail-client.tsx
import Link from 'next/link';
import { StatusBadge } from
  '@/component/task/status-badge';
import { Badge } from '@/component/ui/badge';
import { Card, CardContent, CardHeader, CardTitle } from
  '@/component/ui/card';
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@/component/ui/table';
import { getPriorityBadgeVariant } from
  '@/lib/badge-variant';
import { TASK_PRIORITY_LABELS } from
  '@/lib/constant/priority';
```

ステータス表示は `StatusBadge` に任せます。このコンポーネントが `status` に応じたラベルと色を内部で決めるため、この画面でバッジの見た目を組み立てる必要はありません。優先度は `getPriorityBadgeVariant` で色の種類だけを選び、ラベルは `TASK_PRIORITY_LABELS` から引きます。

右カラムに「参加プロジェクト」カードを追加します。プロジェクト名をリンクにすると、マウスだけでなくキーボードでも移動できます。所属を見て気になったプロジェクトへ、URL を打ち直さずに行けます。

Day 29: ユーザー詳細・編集ページを作ろう

```
    { /* filepath:
src/app/user/[id]/user-detail-client.tsx */
    { /* 参加プロジェクト一覧(バッジ形式) */ }
    <Card>
      <CardHeader>
        <CardTitle className="text-lg">
参加プロジェクト</CardTitle>
      </CardHeader>
      <CardContent>
        {user.projects && user.projects.length >
0 ? (
          <div className="flex flex-wrap gap-2">
            {user.projects.map((member) => (
              <Link
                key={member.id}
```

`Link` の開始タグへ、移動先と見た目を続けます。`href` の `?projectId=` にプロジェクト ID を付けると、移動先は開くプロジェクトを URL から読み取れます。

```
// filepath: src/app/user/[id]/user-detail-client.tsx(同じファイルの続き)

      href={` /project?projectId=${
member.project.id}`}

      className={

'inline-flex items-center gap-2 rounded-md border px-3 py-1 '
+

'text-sm text-foreground hover:bg-muted'

      }
    >
    <span
      aria-hidden="true"
      className="h-2.5 w-2.5
rounded-full"
      style={{ backgroundColor:
member.project.color }}
    />
```

プロジェクトカラーは文字の背景ではなく、小さな丸にだけ使います。名前はテーマに合わせた `text-foreground` で表示するため、白や黄色のプロジェクトカラーでも読めます。

この時点では `<Link>` がまだ閉じていないので画面は出ません。残りを閉じるところまで続けてください。

```
        { /* filepath:
src/app/user/[id]/user-detail-client.tsx(同じファイルの続
き) */}

        {member.project.name}
      </Link>
    )}]
  </div>
) : (
  <p className="text-muted-foreground
text-sm">
    参加しているプロジェクトはありません
  </p>
)
</CardContent>
</Card>
```

) : (から下が、プロジェクトが1件も無い人に出る表示です。

`user.projects.length > 0` の判定が `false` のときにこちらが描かれます。ここでカードが閉じました。外側の箱と `return (` は Step 4 で閉じてあるためファイルはこの時点で通ります。

確認ポイント:

- 右カラムに「参加プロジェクト」のカードが出ます
- `Tab` キーでプロジェクト名へ移動し、`Enter` キーでプロジェクトを開けます

Tailwind CSS では動的な色をクラスで指定できないため、色見本の丸だけ

`style={{ backgroundColor: member.project.color }}` でプロジェクトカラーを適用しています。

「担当中のタスク」カードをテーブル形式で追加します。テーブルのヘッダー部分です。

```

        {/* filepath:
src/app/user/[id]/user-detail-client.tsx */}
        {/* 担当中タスクのテーブル表示 */}
        <Card>
          <CardHeader>
            <CardTitle className="text-lg">
担当中のタスク</CardTitle>
          </CardHeader>
          <CardContent className="p-0">
            {user.assignedTasks &&
user.assignedTasks.length > 0 ? (
              <Table>
                <TableHeader>
                  <TableRow>
                    <TableHead>タイトル</TableHead>
                    <TableHead>ステータス</TableHead>
                    <TableHead>優先度</TableHead>
                    <TableHead>期限</TableHead>
                  </TableRow>
                </TableHeader>

```

確認ポイント: テーブルヘッダーに「タイトル」「ステータス」「優先度」「期限」の4列が表示されることを確認してください。

Day 29: ユーザー詳細・編集ページを作ろう

`TableHeader` の直後に `TableBody` を追加します。各タスクのタイトルをリンクにして、タスク詳細へ移動できるようにします。

```

{/* filepath: src/app/user/[id]/user-detail-client.tsx */}
{/* タスクテーブルのボディ部分 */}

    <TableBody>
      {user.assignedTasks.map((task) => (
        <TableRow key={task.id}>
          <TableCell
            className="font-medium">
              <Link
                href={` /task?taskId=${
task.id}`}
                className="text-primary
underline-offset-4 hover:underline"
              >
                {task.title}
              </Link>
            </TableCell>
            <TableCell>
              <StatusBadge status={
task.status} />
            </TableCell>
            <TableCell>
              <Badge variant={
getPriorityBadgeVariant(task.priority)}>
                {TASK_PRIORITY_LABELS[
task.priority]}
              </Badge>
            </TableCell>

```

Day 29: ユーザー詳細・編集ページを作ろう

リンクにするのはタイトルだけです。`TableRow` にクリック処理を付けると、マウスでは移動できてもキーボードの `Tab` キーでは行を選べません。`Link` ならタイトルへ移動して `Enter` キーで開けます。`key={task.id}` は Day 09 のカード一覧と同じ役目で、React が行の入れ替わりを追うための印です。

期限列の表示とカードの閉じタグを追加します。日付がない場合は `-` を表示します。

```
{/* filepath: src/app/user/[id]/user-detail-client.tsx */}
{/* 期限列とテーブル・カードの閉じタグ */}

      <TableCell>
        {task.dueDate
          ? formatDateOnly(
task.dueDate)
          : '-'}
      </TableCell>
    </TableRow>
  )}
</TableBody>
</Table>
) : (
  <div className="p-6
text-muted-foreground text-sm">
    担当中のタスクはありません
  </div>
)}
</CardContent>
</Card>
```

期限だけ `format` を使わず `formatDateOnly` に任せているのにははっきりした理由があります。`dueDate` は「日付だけ」の値として時刻を UTC の0時にそろえて保存してあります。これを `new Date()` で読み直して各自の時計に合わせると UTC より西の時間帯では前日にずれます。5月10日締切のタスクが5月9日と表示され、期限切れの判定まで1日早まります。`formatDateOnly` は各自の時計へ直さず、UTC のまま日付の部分だけを取り出します。だから誰の時計で見ても同じ日付になります。

確認ポイント: タスクテーブルにステータスバッジと優先度バッジが色付きで表示されることを確認してください。

スクリーンショット: プロジェクト一覧とタスクテーブルの表示を確認してください。

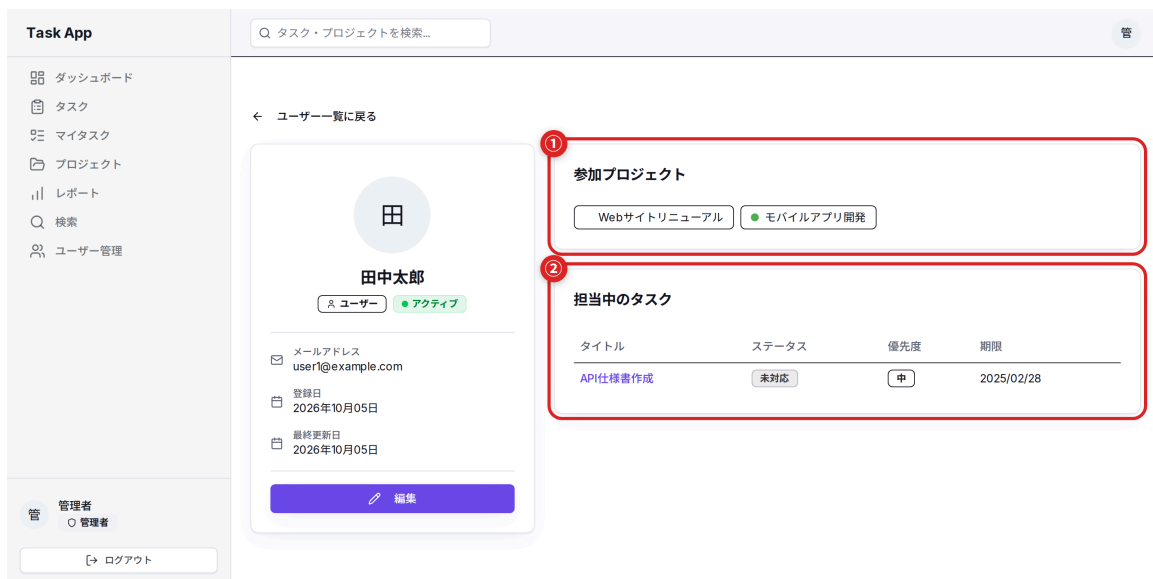


図 4: ユーザー詳細ページ。赤枠①が参加プロジェクトのカード、赤枠②が担当中のタスクのカード

確認ポイント:

- プロジェクトカラーが名前の横の丸に表示されます

Day 29: ユーザー詳細・編集ページを作ろう

- **Tab** と **Enter** でプロジェクト名のリンクを開けます
- **Tab** と **Enter** でタスク名のリンクを開けます
- プロジェクト・タスクがないユーザーには「ありません」メッセージが出ます

Step 6: 権限チェックで編集ボタンを出し分ける(読む目安: 5分)

ゴール: 管理者または本人のみに編集ボタンを表示します。

まずインポートを追加します。

```
// filepath: src/app/user/[id]/user-detail-client.tsx
import { ArrowLeft, Calendar, Mail, Pencil } from
'Lucide-React';
// Pencil を追加(ArrowLeft, Calendar, Mail は Step 4 で追加済み)
```

アイコンはすべて **Lucide-React** から取り込むので行を増やさず Step 4 で書いた行へ **Pencil** を足します。まとめて1行にしておくとそのファイルが使うアイコンを1か所で見渡せます。ボタンを置く場所は左カラムの下端です。プロフィールを読んだから操作へ進む順番になります。

```

      { /* filepath:
src/app/user/[id]/user-detail-client.tsx */
      {(isAdmin || isOwnProfile) && (
        <>
          <Separator className="my-4" />
          <Button
            className="w-full"
            onClick={() =>
              router.push(`/user/${
user.id}/edit`)
            }
          >
            <Pencil className="mr-2 h-4 w-4"
/> 編集
          </Button>
        </>
      )}

```

条件	結果
管理者 + 他人のプロフィール	ボタンが表示されます
管理者 + 自分のプロフィール	ボタンが表示されます
一般ユーザー + 自分のプロフィール	ボタンが表示されます
一般ユーザー + 他人のプロフィール	そもそもページにアクセスできません (サーバーが FORBIDDEN を返します)

一般ユーザーが他人のユーザー ID で `/user/他人のid` にアクセスすると `getById` の権限チェックで FORBIDDEN エラーになり、データが取得できません。「ボタンが非表示」ではなく「ページ自体が表示されない」という設計です。

確認ポイント:

- 管理者でログインするとどのユーザーページにも「編集」ボタンが表示されます
- 一般ユーザーで自分のページを見るとボタンが表示されます
- 一般ユーザーで他人の ID にアクセスすると「このユーザーを見る権限がありません」のカードが表示され、ユーザー情報は表示されません

Step 7: 編集ページのファイルを作成する(読む目安: 5 分)

ゴール: `/user/[id]/edit` の入口とフォームの骨組みを作り、取得失敗で待ち続けない画面にします。

詳細ページと同じく、編集ページもブラウザへ渡す前に `getById` で認証・認可を確認します。

```
// filepath: src/app/user/[id]/edit/page.tsx
import { getTRPCErrorFromUnknown } from '@trpc/server';
import { notFound, redirect } from 'next/navigation';
import { AppLayout } from '@component/layout/app-layout';
import { Card, CardContent } from '@component/ui/card';
import { trpc } from '@trpc/server';
import { UserEditClient } from './user-edit-client';

interface UserEditPageProps {
  params: Promise<{
    id: string;
  }>;
}

export default async function UserEditPage({ params }:
UserEditPageProps) {
  const { id } = await params;

  try {
    await trpc.user.getById({ id });
  } catch (error) {
    const trpcError = getTRPCErrorFromUnknown(error);

```

編集側の server wrapper は、URLのIDを client へ渡す前に `getById` を呼びます。画面の表示後に client だけで権限を確かめると、拒否される人にもフォームの枠が一瞬表示されるからです。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/page.tsx(同じファイルの続き)

if (trpcError.code === 'UNAUTHORIZED')
  redirect('/login');

if (trpcError.code === 'NOT_FOUND') notFound();

if (trpcError.code === 'FORBIDDEN') {
  return (
    <AppLayout>
      <div className="container mx-auto max-w-md mt-8">
        <Card>
          <CardContent className="pt-6">
            <h1 className="text-xl font-bold mb-2">
              アクセス権がありません
            </h1>
            <p className="text-muted-foreground">
              管理者または本人のみユーザー編集が可能です
            </p>
          </CardContent>
        </Card>
      </div>
    </AppLayout>
  );
}
```

拒否画面はユーザーがとれる行動を明示します。他人の实在IDと存在しないIDで同じ表示にすると、権限のない人が画面の違いからユーザーの实在を推測できません。

```
// filepath: src/app/user/[id]/edit/page.tsx(同じファイルの続き)

    throw error;
  }

  return <UserEditClient userId={id} />;
}
```

詳細ページを通らず編集 URL を直接開くこともできます。そのため編集側にも同じ判定が必要です。一般ユーザーが他人の存在 ID と存在しない ID を開いた場合は、どちらも **FORBIDDEN** の拒否画面になります。権限のある人にだけ **NOT_FOUND** の 404 を返します。

次に client 側の取り込み、スキーマ、フォーム初期値を書きます。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { ArrowLeft } from 'lucide-react';
import { useRouter } from 'next/navigation';
import { useEffect, useRef } from 'react';
import { useForm } from 'react-hook-form';
import toast from 'react-hot-toast';
import { z } from 'zod';
import { AppLayout } from '@component/layout/app-layout';
import { Button } from '@component/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from
 '@component/ui/card';
import { PageLoadingSpinner } from
 '@component/ui/loading-spinner';
import { USER_ROLE } from '@lib/constant/roles';
import {
  httpStatusOf,
  isAuthError,
  isForbiddenError,
  shouldRetryQuery,
} from '@lib/query-error';
import { api } from '@trpc/react';
```

ArrowLeft は戻るボタン、4つの query helper は取得結果の分類に使います。

toast は Step 10 の保存結果で使うため、この時点で取り込みます。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)
```

```
const shouldRetryUserQuery = (failureCount: number,
error: unknown) =>
```

```
  httpStatusOf(error) !== 404 && shouldRetryQuery(
failureCount, error);
```

```
const USER_ROLE_VALUES = ['USER', 'ADMIN'] as const;
```

```
const userEditSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  avatar:
z.string().url('有効なURLを入力してください').or(
z.literal('')),
  role: z.enum(USER_ROLE_VALUES),
  isActive: z.boolean(),
});
```

```
type UserEditFormValues = z.infer<typeof userEditSchema>;
```

```
interface UserEditClientProps {
  userId: string;
}
```

```
export function UserEditClient({ userId }:
UserEditClientProps) {
  const router = useRouter();

  const form = useForm<UserEditFormValues>({
    resolver: zodResolver(userEditSchema),
```

Day 29: ユーザー詳細・編集ページを作ろう

文字列の配列へ `as const` を付けると、zod と TypeScript の両方がロールを `'USER' | 'ADMIN'` の2択として扱えます。アバターは有効なURLか空文字だけを許します。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

defaultValues: {
  name: '',
  avatar: '',
  role: USER_ROLE.USER,
  isActive: true,
},
});

const {
  data: currentUser,
  isLoading: isCurrentUserLoading,
  isError: isCurrentUserError,
  isFetching: isCurrentUserFetching,
  error: currentUserError,
  refetch: refetchCurrentUser,
} = api.auth.getCurrentUser.useQuery(
  undefined,
  { retry: shouldRetryQuery },
);

const isAdmin = currentUser?.role === USER_ROLE.ADMIN;
const isOwnProfile = currentUser?.id === userId;
const canEditUser = isAdmin || isOwnProfile;
const canManageAccount = isAdmin && !isOwnProfile;
```

ここでは編集対象の `user` が届く前に query の `enabled` を決めるため、本人判定にURLから受け取った `userId` を使います。`user.id` を使うと、取得前は本人と判定できず query 自体が始まりません。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

const {
  data: user,
  isLoading: isUserLoading,
  isError: isUserError,
  isFetching: isUserFetching,
  error: userError,
  refetch: refetchUser,
} = api.user.getById.useQuery(
  { id: userId },
  {
    enabled: !!currentUser && canEditUser &&
    userId.length > 0,
    retry: shouldRetryUserQuery,
  },
);
```

一般ユーザーが他人を開いた場合は `enabled` が `false` になり、対象データを取りに行きません。サーバー側の先行確認も同じアクセスを拒否します。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)
```

```
const submitContextRef = useRef({
  targetUserId: userId,
  user,
  canEditUser,
  canManageAccount,
});

submitContextRef.current = {
  targetUserId: userId,
  user,
  canEditUser,
  canManageAccount,
};

const queryErrors = [
  isCurrentUserError ? currentUserError : null,
  isUserError ? userError : null,
];

const authFailed = queryErrors.some(isAuthError);
const forbidden = queryErrors.some(isForbiddenError);
const notFound = isUserError && httpStatusOf(userError)
=== 404;
```

submitContextRef には最新のURLのID、取得した利用者、権限をひとまとめに保存します。画面Aで作られた送信処理が画面Bへ移動した後に遅れて呼ばれても、AのIDへBの入力値を送らないためです。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

const hasFetchError = isCurrentUserError || isUserError;

const hasRequiredData =
  (!isCurrentUserError || currentUser !== null)
  && (!isUserError || user !== null);

const requiredLoading =
  isCurrentUserLoading || (canEditUser &&
isUserLoading);

const requiredFetching = isCurrentUserFetching ||
isUserFetching;

const refetchRequiredData = () => {
  void refetchCurrentUser();

  if (canEditUser) void refetchUser();
};

if (requiredLoading && !authFailed && !forbidden
&& !notFound) {
  return (
    <AppLayout>
      <PageLoadingSpinner />
    </AppLayout>
  );
}
```

スピナーを出すのは query が動いている間だけです。取得が終わってデータがない場合は次のエラー画面へ進みます。取得済みかどうかを `isLoading` だけではなく必須データの有無と一緒に判定します。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

if (
  authFailed
  || forbidden
  || notFound
  || (hasFetchError && !hasRequiredData)
  || !currentUser
) {
  return (
    <AppLayout>
      <div className="flex flex-col items-center
justify-center py-24 text-center">
        <p className="mb-2 text-base font-semibold
text-foreground">
          {authFailed
            ? 'ログインの有効期限が切れました'
            : forbidden
            ? 'このユーザーを編集する権限がありません'
            : notFound
            ? 'ユーザーが見つかりません'
            : 'ユーザー情報を取得できませんでした'}
        </p>
        <p className="mb-6 text-sm
text-muted-foreground">
```

この分岐では、取得に必要なデータがそろわない状態をまとめて画面から外します。原因に応じた見出しを出します。入力フォームやキャッシュされた個人情報を、拒否画面の裏に残さないためです。

```

{/* filepath:
src/app/user/[id]/edit/user-edit-client.tsx(同じファイル
の続き) */}

    {authFailed
      ? 'もう一度ログインしてください。'
      : forbidden
      ? '管理者または本人のみ編集できます。'
      : notFound
      ?
        '削除されたか、URLが正しくない可能性があります。'
        :
        '通信状況を確認して、再読み込みしてください。'}

  </p>
  <Button
    type="button"
    onClick={() => {
      if (authFailed) {
        router.push('/login');
        return;
      }
      if (forbidden) {
        router.push('/dashboard');
        return;
      }
      if (notFound) {

```

Day 29: ユーザー詳細・編集ページを作ろう

```
router.push(isAdmin ? '/user' :  
  '/dashboard');  
  return;  
}
```

状態ごとの行き先をボタンのラベルと合わせます。たとえば「ログイン画面へ」と表示しながら再取得すると、押した結果と文言が食い違います。見出し、説明、ボタンの3つを1つの状態としてそろえます。

```

{ /* filepath:
src/app/user/[id]/edit/user-edit-client.tsx(同じファイル
の続き) */

    refetchRequiredData();
  }}
  disabled={requiredFetching}
>
  {authFailed
    ? 'ログイン画面へ'
    : forbidden
    ? 'ダッシュボードへ'
    : notFound
    ? isAdmin
      ? 'ユーザー一覧へ'
      : 'ダッシュボードへ'
    : '再読み込み'}

  </Button>
</div>
</AppLayout>
);
}

```

401と403ではキャッシュに前回データがあってもフォームを隠します。初回の通信失敗は再読み込みを案内します。500相当の一時失敗でも前回データが残っている場合はフォームを保ち、後で追加する警告を表示します。

Day 29: ユーザー詳細・編集ページを作ろう

```
{/* filepath:
src/app/user/[id]/edit/user-edit-client.tsx(同じファイル
の続き) */}

if (!canEditUser) {
  return (
    <AppLayout>
      <div className="container mx-auto max-w-md mt-8">
        <Card>
          <CardContent className="pt-6">
            <h1 className="text-xl font-bold mb-2">
              アクセス権がありません
            </h1>
            <p className="text-muted-foreground">
              管理者または本人のみユーザー編集が可能です
            </p>
          </CardContent>
        </Card>
      </div>
    </AppLayout>
  );
}
```

この分岐は client 側の二重チェックです。server wrapper が先に拒否しますが、画面を開いた後にセッションや権限が変わる場合もあります。ブラウザ側でも `canEditUser` を確かめ、フォーム本体へ進ませません。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

if (!user) {

  return (

    <AppLayout>

      <div className="flex flex-col items-center
justify-center py-24 text-center">

        <p className="mb-2 text-base font-semibold
text-foreground">

          ユーザー情報を取得できませんでした

        </p>

        <p className="mb-6 text-sm
text-muted-foreground">

          通信状況を確認して、再読み込みしてください。

        </p>

        <Button onClick={refetchRequiredData} disabled={
requiredFetching}>

          再読み込み

        </Button>

      </div>

    </AppLayout>

  );

}
```

`!user` をスピナーへ戻さないことが大切です。待機中ではないのにデータがない状態でスピナーを出すと、失敗しても画面が永久に変わりません。

権限チェックを通過した後に編集フォームの枠を表示します。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

return (
  <AppLayout>
    <div className="container mx-auto max-w-md mt-8 mb-8">
      {hasFetchError && (
        <div role="alert" className="mb-4 rounded-lg border px-4 py-3">
          <span>
            最新のユーザー情報を取得できませんでした。
            入力内容は保持されています。
          </span>
          <Button
            type="button"
            onClick={refetchRequiredData}
            disabled={requiredFetching}
          >
            再試行
          </Button>
        </div>
      )}
      <Button
        variant="ghost"
        onClick={() => router.push(`/user/${userId}`)}
      >
```

この警告は説明を読みやすくするため className を短くした途中版です。完成コードでは配色と間隔も加えます。キャッシュがある 500 相当の失敗ではフォームと入力内容を残し、再試行できるようにします。

```
{/* filepath:
src/app/user/[id]/edit/user-edit-client.tsx(同じファイル
の続き) */}

    <ArrowLeft className="mr-2 h-4 w-4" />
    戻る
  </Button>

  <Card>
    <CardHeader>
      <CardTitle>ユーザー編集</CardTitle>
    </CardHeader>
    <CardContent>
      {/* Step 8, 9, 10 でフォームを追加 */}
    </CardContent>
  </Card>
</div>
</AppLayout>
);
}
```

戻るボタンとフォームの枠をここで閉じると、Step 7 の時点でも JSX 全体の対応関係が確かめられます。後続の Step 8~10 では `CardContent` の中だけをフォームに置き換えます。ページ全体の閉じタグに触れず進められます。

確認ポイント:

- 存在しないユーザー ID では、権限のある人だけ `notFound()` の 404 になります
 - 権限のない一般ユーザーは、実在 ID と存在しない ID のどちらを開いても同じ拒否画面になります
 - 初回取得失敗では再読み込み、401 ではログイン、403 ではダッシュボードへのボタンが出ます
 - 読み込み中だけスピナーが表示され、取得失敗後に回り続けません
 - 「戻る」ボタンで詳細ページに戻れます
-

Step 8: zod スキーマと useForm でデータを同期する(読む目安: 7 分)

ゴール: react-hook-form + zod でフォームを管理し、サーバーデータを自動入力します。

zod スキーマと `useForm` は Step 7 で定義済みです。ここでは重複して追加せず、`z.infer` で型が自動生成されていることを確認してください。

`useEffect` + `form.reset` でサーバーデータが届いたらフォームに反映します。同じ利用者の再取得では入力途中の項目を守り、まだ触っていない項目だけを最新値へ更新します。

次のコードは `UserEditClient` 関数内へ入れます。位置は Step 7 の `submitContextRef.current = { ... };` の代入直後、
`const queryErrors =` の直前です。この位置では `user` の取得処理が先に書かれているため、effect の依存配列で `user` を参照できます。早期リターンより前に置き、描画ごとにフックを同じ順番で呼びます。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
const { dirtyFields } = form.formState;
const dirtyFieldsRef = useRef(dirtyFields);
dirtyFieldsRef.current = dirtyFields;
const hydratedUserIdRef = useRef<string | null>(null);

useEffect(() => {
  if (user) {
    const sameUser = hydratedUserIdRef.current ===
user.id;
    form.reset(
      {
        name: user.name ?? '',
        avatar: user.avatar ?? '',
        role: user.role,
        isActive: user.isActive,
      },
      sameUser && Object.keys(
dirtyFieldsRef.current).length > 0
        ? { keepDirtyValues: true } : undefined,
    );
    hydratedUserIdRef.current = user.id;
  }
}, [user, form]);
```

Day 29: ユーザー詳細・編集ページを作ろう

`dirtyFields` (利用者が変更した項目の記録)を読むと、`keepDirtyValues` が入力途中の値を守れます。`dirtyFieldsRef` は再取得の effect を増やさずに最新の記録を参照するための箱です。初回表示と別の利用者への移動では全項目を入れ直します。同じ利用者の再取得では変更した項目を残し、未変更のロールや有効状態などをサーバーの最新値へ更新します。

確認ポイント: ファイルを保存してエラーが出ないことを確認してください。

次に必要なコンポーネントをインポートし、フォームを書きます。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
import { Avatar, AvatarFallback, AvatarImage }
  from '@component/ui/avatar';
import { Input } from '@component/ui/input';
import { Label } from '@component/ui/label';
```

CardContent 内のフォームを書きます。`register` でテキスト入力を管理し、`watch` でアバタープレビューをリアルタイム更新します。

`onSubmit` と `updateUser` は Step 10 で定義します。ここでは JSX 構造を先に書き、Step 10 の完了後に動作確認します。

```

{/* filepath: src/app/user/[id]/edit/user-edit-client.tsx
*/}
{/* フォーム開始 + アバタープレビュー */}
    <form onSubmit={
      form.handleSubmit(onSubmit)}
      className="space-y-6">
      <div className="flex
        justify-center mb-6">
        <Avatar className="w-24 h-24">
          {form.watch('avatar') && (
            <AvatarImage
              src={form.watch('avatar')}
              alt="" />
          )}
        <AvatarFallback
          className="text-2xl">
            {form.watch('name')
              ?. [0]?.toUpperCase()}
          </AvatarFallback>
        </Avatar>
      </div>

```

`form.watch('avatar')` はその項目の今の入力値を読み出して変わるたびに描き直させる書き方です。だからURLを1文字打つごとにプレビューが差し替わります。`AvatarImage` を `form.watch('avatar') &&` で囲むのは空欄のときに `src=""` の画像を出さないためです。空欄なら下の `AvatarFallback` が名前の頭文字を表示します。

```
{/* filepath: src/app/user/[id]/edit/user-edit-client.tsx
*/}
{/* 名前入力(register版) */}

    <div className="space-y-2">
      <Label htmlFor="name">
        名前
        <span className=
          "text-destructive">*
        </span>
      </Label>
      <Input id="name"
        {...form.register('name')}
        disabled={
          updateUser.isPending} />
      {form.formState.errors
        .name && (
          <p className="text-sm
            text-destructive">
              {form.formState.errors
                .name.message}
            </p>)}
    </div>
```

`{...form.register('name')}` の1行で、この入力欄が `useForm` の管理下に入ります。`register` は `name`・`onChange`・`onBlur`・`ref` を渡し、入力値を読み取れるようにします。下の `form.formState.errors.name` はStep 7 の zod スキーマに書いた「名前を入力してください」を受け取る場所です。文言をスキーマ側だけに置けるので検査の条件と画面の表示が食い違いません。

メールアドレスとアバターURL入力欄を追加します。

```
    { /* filepath:
src/app/user/[id]/edit/user-edit-client.tsx */}

    <div className="space-y-2">
      <Label htmlFor="email">
        メールアドレス</Label>
      <Input id="email"
        value={user.email}
        disabled />
      <p className="text-xs
        text-muted-foreground">
        メールアドレスは変更できません
      </p>
    </div>

    <div className="space-y-2">
      <Label htmlFor="avatar">
        アバターURL(任意)</Label>
      <Input id="avatar" type="url"
        {...form.register('avatar')}
        disabled={
          updateUser.isPending}

placeholder="https://example.com/avatar.png"
      />
    </div>
```

Day 29: ユーザー詳細・編集ページを作ろう

メールアドレスの欄だけ `register` を使わず、`user.email` を直接入れて `disabled` にしています。この欄は送信の対象ではないからです。Step 7 の `userEditSchema` にも `email` は入れていません。ログインに使う値なのでここで気軽に書き換えられると本人が締め出されます。それでも欄ごと消さないのはいま誰を編集しているのかを画面で確かめられるようにするためです。読み取り専用で置いておくと開くページを間違えたことにその場で気付けます。

画面での確認は Step 10 の最後で行います。ここではコードを見ます。

確認ポイント:

- `<form onSubmit={form.handleSubmit(onSubmit)}>` から名前・メール・アバターの3つの `<div>` までが書けています
- 名前欄に `{...form.register('name')}` が入っています
- メールアドレスの欄だけ `register` を使わず `value={user.email}` と `disabled` になっています

Step 9: ロール選択・アクティブ状態の切り替えを実装する(読む目安: 7分)

ゴール: Select でロールを選び、Checkbox でアクティブ状態を切り替えられるようにします。

サーバー側のアクセス制御: 本人は管理者であっても自分の `role` や `isActive` を変更できません。Step 10 では `canManageAccount` を使い、管理者が他人を編集するときだけ送信します。

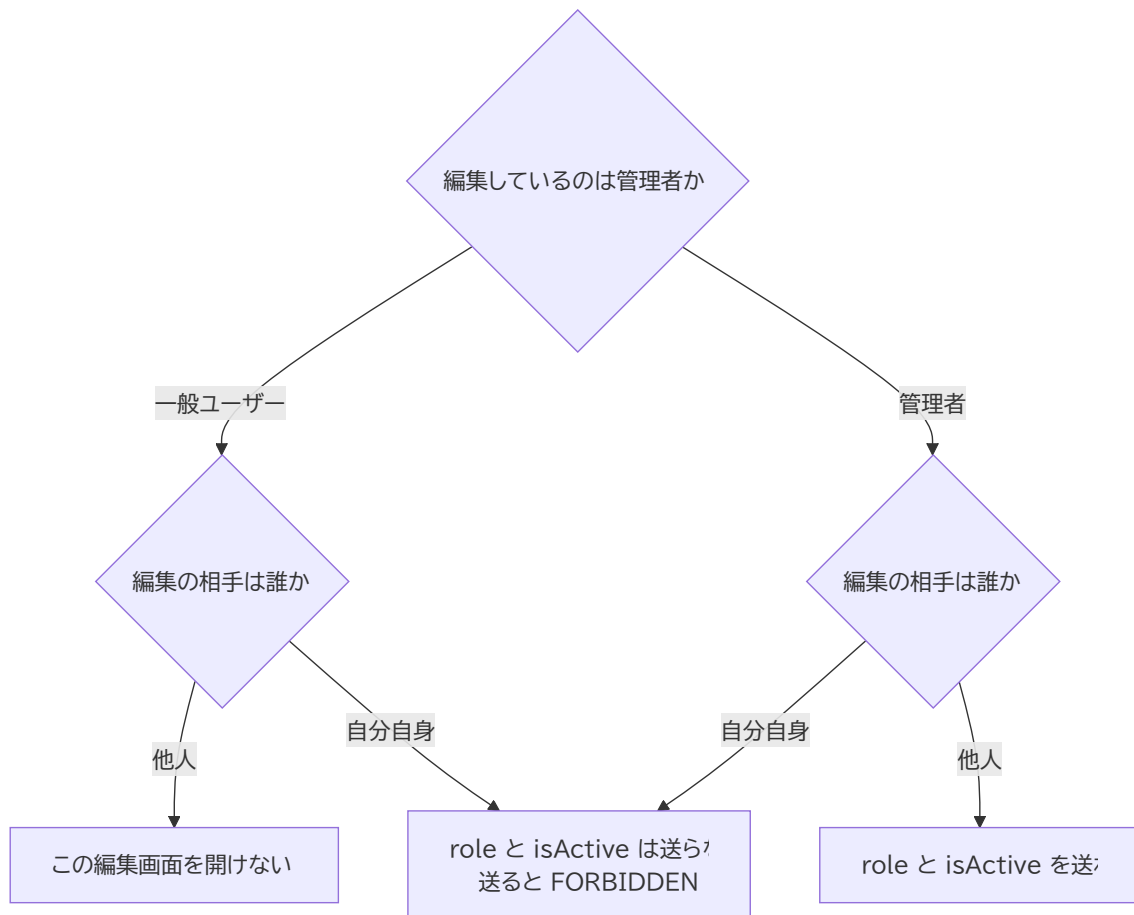


図5: Step 9: ロール選択・アクティブ状態の切り替えを実装する

管理者かどうかだけでは決まらず、相手が誰かでもう一度分かります。右下の枝があるのは管理者が自分の権限を下げたり自分を停止したりして誰も管理できなくなる状態を防ぐためです。

インポートを追加します。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
import { Checkbox } from '@component/ui/checkbox';
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from '@component/ui/select';
import { isUserRole, USER_ROLE_LABELS }
  from '@lib/constant/roles';
```

確認ポイント: 取り込んだ部品の名前が赤くなっていないことを確認してください。

`</form>` はこのあと書くのでこの時点では閉じタグが足りないというエラーが残ります。`useMutation` は Step 10 で追加します。

アバター URL 入力欄の直後に、ロール選択とアクティブ切り替えを追加します。ここがフォームの末尾になります。

次のロール選択とアクティブ切り替えは `canManageAccount` が `true` の場合だけ表示します。

```
{/* filepath: src/app/user/[id]/edit/user-edit-client.tsx
*/}
{canManageAccount && (
  <>
```

`canManageAccount` は Step 7 で `isAdmin && !isOwnProfile` として作った値です。管理者が自分自身を編集しているときは `false` になるのでこの囲みの中は画面に出ません。管理者が自分の権限を自分で下げてしまい、誰も管理できないアプリになる事故を防ぐためです。`<>` は画面に何も残さない囲みで、条件が成り立つときだけ中の2つをまとめて出します。

```
{/* filepath: src/app/user/[id]/edit/user-edit-client.tsx
*/}
{/* ロール選択(form.setValue版) */}
    <div className="space-y-2">
      <Label htmlFor="role">
        ロール</Label>
      <Select
        value={form.watch('role')}
        onChange={(value) => {
          if (isUserRole(value)) {
            form.setValue(
              'role', value, { shouldDirty:
true });
          }
        }}
        disabled={
          updateUser.isPending}>
        <SelectTrigger id="role">
          <SelectValue
            placeholder=
              "ロールを選択" />
        </SelectTrigger>
```

Day 29: ユーザー詳細・編集ページを作ろう

ここだけ `register` を使わず、`value` と `onValueChange` の2つで `useForm` へつないでいます。`shouldDirty: true` は選んだロールを変更済みの項目として記録します。再取得時の `keepDirtyValues` がその選択を保持するために必要です。`Select` は素の `<select>` タグではなく、ボタンと一覧を組み合わせて作られた部品なので `register` が待っている `onChange` が発生しないからです。代わりに `form.setValue` で値を書き戻します。選択肢の中身は次のブロックで足すのでいまは開いたままにしておきます。

```
{/* filepath: src/app/user/[id]/edit/user-edit-client.tsx
*/}
{/* ロール選択肢(SelectContent) */}
    <SelectContent>
      {Object.entries(
        USER_ROLE_LABELS
      ).map(([value, label]) => (
        <SelectItem
          key={value}
          value={value}>
            {label}
          </SelectItem>
        ))}
    </SelectContent>
  </Select>
</div>
```

確認ポイント: `SelectItem` が「ユーザー」「管理者」の2つ書けていることをコードで確認してください。画面での確認は `</form>` を書き終える Step 10 の最後で行います。

`onValueChange` が渡してくる `value` はどんな文字列でもありうる `string` 型です。一方 `form.watch('role')` の型は zod スキーマから作られるので `'USER' | 'ADMIN'` の2択に絞られています。広いほうの型を狭いほうへそのまま入れることはできないので `isUserRole` 型ガードで中身を確認してから渡します (`as UserRole` は使いません)。

アクティブ状態を切り替えるチェックボックスを追加します。

```

    { /* filepath:
src/app/user/[id]/edit/user-edit-client.tsx */ }

    <div className="flex
      items-center space-x-2">
      <Checkbox id="isActive"
        checked={form.watch(
          'isActive')}
        onChange={
          (checked) =>
            form.setValue(
              'isActive',
              checked === true, { shouldDirty:
true })}
        disabled={
          updateUser.isPending} />
      <Label htmlFor="isActive">
        アクティブ</Label>
    </div>

```

Day 29: ユーザー詳細・編集ページを作ろう

`checked` と `onCheckedChange` の組み方は上の `Select` とそろえてあります。ここでも `shouldDirty: true` を渡し、再取得で入力途中の有効状態を上書きしないようにします。`form.watch('isActive')` で今の値を映し、切り替わったら `form.setValue` で書き戻す形です。どの入力欄にも `disabled={updateUser.isPending}` を付けているのは保存中に値を変えられると送った内容と画面に見えている内容が食い違うからです。

```
    {/* filepath:
src/app/user/[id]/edit/user-edit-client.tsx */}
    </>
  )}
```

確認ポイント: `Checkbox` に `checked` と `onCheckedChange` の2つが書けていることをコードで確認してください。実際の切り替えは Step 10 の最後で確かめます。

`checked === true` と書くのは `onCheckedChange` が `boolean | 'indeterminate'` を受け取るためです。

スクリーンショット: 完成後の編集フォームはこのように表示されます(この時点ではまだ描画されません)。



The image shows a 'User Edit' form with a light gray background and rounded corners. At the top, the title 'ユーザー編集' is in bold. Below it is a circular avatar placeholder with the character '田' in the center. The form contains several input fields: '名前*' (Name) with the value '田中太郎', 'メールアドレス' (Email Address) with the value 'user1@example.com' and a note 'メールアドレスは変更できません' (Email address cannot be changed), and 'アバターURL (任意)' (Avatar URL (Optional)) with the value 'https://example.com/avatar.png'. There is a 'ロール' (Role) dropdown menu currently showing 'ユーザー'. Below the dropdown is a checked checkbox labeled 'アクティブ' (Active). At the bottom, there are two buttons: a purple '更新' (Update) button and a white 'キャンセル' (Cancel) button.

図6: ユーザー編集フォーム。名前とメールアドレスの下に、ロールを選ぶ欄と「アクティブ」のチェックボックスが並んでいる

画面での確認は Step 10 の最後で行います。ここではコードを見ます。

確認ポイント:

- ロール選択の `SelectItem` に「ユーザー」「管理者」の2つが書けています
- `Select` の `value` に現在のロールが渡っています
- `Checkbox` に `checked` と `onCheckedChange` が書けています

Step 10: 保存機能を実装して完成(読む目安: 5分)

ゴール: 保存ボタンを押すと tRPC で DB が更新され、詳細ページに戻るところまで作ります。

Day 25 で学んだ `useMutation` パターンを使い、保存処理を実装します。

`useMutation` は `UserEditClient` 関数内の早期リターンより前へ追加します。読み込み中や取得失敗時も、フックは毎回同じ順番で呼ぶ必要があるためです。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
import { AlertCircle } from 'lucide-react';
import { Alert, AlertDescription, AlertTitle }
  from '@component/ui/alert';
// ArrowLeft は既にインポート済みなので AlertCircle を追加
```

`Alert` を足すのは保存に失敗した理由を画面に残すためです。この後に書く `onError` のトーストは数秒で消えるので目を離している間に失敗すると読者は何が起きたか分かりません。消えない場所にも同じ内容を出しておきます。

確認ポイント: 取り込んだ部品の名前が赤くなっていないことを確認してください。

`</form>` はこのあと書くのでこの時点では閉じタグが足りないというエラーが残ります。

`api.useUtils()` で、更新後に再取得する 4 つの query の呼び出し口を用意します。`updateUser` と同じく早期リターンより前へ置いてください。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
const utils = api.useUtils();

const updateUser =
  api.user.update.useMutation({
    onSuccess: async () => {
      await Promise.allSettled([
        utils.user.getById.invalidate(
          { id: userId }),
        utils.user.getAll.invalidate(),
        utils.auth.getCurrentUser.invalidate(),
        utils.auth.getSession.invalidate(),
      ]);
      toast.success(
        'ユーザー情報を更新しました');
      router.push(`/user/${userId}`);
      router.refresh();
    },
    onError: (error) => {
      toast.error(error.message
        ?? 'ユーザー情報の更新に'
        + '失敗しました');
    },
  });
```

`onSuccess` で4つを `invalidate` しているのは同じ人の情報を4か所が別々に覚えているからです。詳細ページの `getById`、一覧の `getAll`、サイドバーに名前を出す `getCurrentUser`、そしてセッションです。名前を変えたのに一覧やサイ

Day 29: ユーザー詳細・編集ページを作ろう

ドバーだけ古いままになるのはここを1つ書き忘れたときに起きます。

`Promise.allSettled` でまとめてあるのでどれか1つが失敗しても残りの取り直しは進み、通知と画面遷移までたどり着きます。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx
// zodバリデーション済みの値で送信
const onSubmit =
  (values: UserEditFormValues) => {
    const submitContext =
      submitContextRef.current;
    if (
      !submitContext.user
      || !submitContext.canEditUser
      || submitContext.targetUserId !== userId
      || submitContext.user.id !== userId
    ) return;

    updateUser.mutate({
      id: userId,
      name: values.name,
      avatar: values.avatar
      || null,
```

URLの `userId`、最新の送信対象ID、取得済みの `user.id` の3つがそろわない場合は送信しません。画面遷移と送信が重なっても、別の利用者に入力値を送らないための確認です。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx(同じファイルの続き)

...(submitContext.canManageAccount
  && values.role
    !== submitContext.user.role
  ? { role: values.role }
  : {}),
...(submitContext.canManageAccount
  && values.isActive
    !== submitContext.user.isActive
  ? { isActive: values.isActive }
  : {}),
});
};
```

確認ポイント: `onSubmit` と `updateUser` が定義できた。`</form>` はこのあと書くのでこの時点ではまだ構文エラーが残ります。これで Step 8 で書いた `<form onSubmit={form.handleSubmit(onSubmit)}>` が指している2つがそろいました。実際に動くかどうかは `</form>` を書き終えた Step 10 の最後で確かめます。

サーバー側の `update` ルーターは自分のプロフィール更新で `role` や `isActive` が含まれると `FORBIDDEN` を返します。管理者が他人を編集する場合も、読み込んだ値から変えた `role` と `isActive` だけを追加します。同じ値を送り直すと、名前だけの更新でセッション版数を増やしたり、別の管理者が変更した有効状態を古いフォームから戻したりするためです。`avatar` の空文字は `null` に変換し、登録済みの画像を消せる送信にします。

エラー表示ブロックをチェックボックスの下に追加します。

```
      {/* filepath:
src/app/user/[id]/edit/user-edit-client.tsx */}
      {updateUser.error && (
        <Alert variant="destructive">
          <AlertCircle className="h-4 w-4" />
          <AlertTitle>エラー</AlertTitle>
          <AlertDescription>
            {updateUser.error.message}
          </AlertDescription>
        </Alert>
      )}
```

確認ポイント: `{updateUser.error && (...)}` のブロックがチェックボックスの下に書けていることを確認してください。赤いアラートの実表示は `</form>` を書き終えた後の最終確認で見ます。

`toast` は一時的な通知(数秒で消える)、`Alert` はフォーム内に残り続ける表示です。両方使うことでユーザーにエラーを確実に伝えます。

フォームの送信ボタンとキャンセルボタンを追加します。

```

        { /* filepath:
src/app/user/[id]/edit/user-edit-client.tsx */}
        <div className="flex gap-2 pt-2">
            <Button
                type="submit"
                className="w-full"
                disabled={updateUser.isPending}
            >
                {updateUser.isPending ? '更新中...' :
'更新'}
            </Button>
            <Button
                type="button"
                variant="outline"
                className="w-full"
                onClick={() => router.push(`/user/${
userId}`)}
                disabled={updateUser.isPending}
            >
                キャンセル
            </Button>
        </div>
    </form>

```

キャンセル側の `type="button"` は書き忘れると事故になる1行です。

`<button>` はフォームの中に置くと何も指定しない限り送信ボタンとして扱われます。つまり `type` を書かずにいるとキャンセルを押した瞬間に保存が走ります。取り

Day 29: ユーザー詳細・編集ページを作ろう

消すつもりの操作が保存になるので読者は自分が何をしたのか分かりません。2つのボタンに `disabled={updateUser.isPending}` を付けているのも同じ考え方で、保存中の連打で同じ更新が二重に飛ぶのを止めます。

確認ポイント:

- フォームに変更を加えて「更新」をクリックするとトースト通知が表示されます
- 保存成功後、詳細ページへ戻り変更内容が反映されています
- 保存中はボタンが「更新中...」に変わりグレーアウトします
- 「キャンセル」をクリックすると詳細ページに戻ります(変更は保存されません)
- `npm run dev` でエラーが出ません
- 編集ページを開くとフォームにユーザーの名前が自動入力されています
- アバターURLを入力するとプレビューがリアルタイムで変わります
- メールアドレスの入力欄がグレーアウトして編集できません
- 自分のページを開いたときはロールとアクティブの欄が出ません
- 管理者で他人のページを開くとロールとアクティブの欄が出ます
- ロール選択には「ユーザー」「管理者」が並びます
- アクティブのチェックボックスは ON/OFF を切り替えられます

ロールとアクティブの欄は `canManageAccount`

(`isAdmin && !isOwnProfile`)の中にあります。一般ユーザーには欄そのものが出ず、送信データにも `role` と `isActive` が入りません。サーバーの `FORBIDDEN` は最後の守りで、画面からは届かない形になっています。

Pro パターンで書こう(ユーザー編集フォームは zod で境界バリデーションする)

入力境界に zod を置くと `role` などのフィールドに不正値が混入したときに実行時エラーとして検出できます。なぜ直前の1文の書き方をするのか、**Before/After**で見比べてみましょう。

Day 29: ユーザー詳細・編集ページを作ろう

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)
import { USER_ROLE, type UserRole } from
'@/lib/constant/roles';

type UserEditFormValues = {
  name: string;
  avatar: string;
  role: UserRole;
  isActive: boolean;
};

type UpdateUserInput = {
  id: string;
  name: string;
  avatar?: string;
  role?: UserRole;
  isActive?: boolean;
};

type UpdateUserMutation = {
  mutate: (input: UpdateUserInput) => void;
};

export function submitUserEditForm(
  rawValues: unknown,
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

入口の引数が `rawValues: unknown` になっているところに目を留めてください。`unknown` は「まだ何の型か分からない値」を表し、そのままでは中身に触れません。フォームから来た値を最初に受け止める型としてはこれで正しいです。Before と After が分かれるのはこの `unknown` をこの先どう扱うかという1点です。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

userId: string,
updateUser: UpdateUserMutation,
isAdmin: boolean,
) {
  const values = rawValues as UserEditFormValues;

  updateUser.mutate({
    id: userId,
    name: values.name,
    avatar: values.avatar || undefined,
    ...(isAdmin
      ? {
        role: values.role,
        isActive: values.isActive,
      }
      : {}),
  });
}

submitUserEditForm(
  {
    name: 'Taro',
    avatar: '',
    role: USER_ROLE.ADMIN,
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

問題は `const values = rawValues as UserEditFormValues;` の1行です。`as` は値を1つも見ずに「この形だと思って進む」と伝える書き方なので `unknown` で受け取った意味がここで消えます。呼び出し側が正しい形を渡している限りは動きます。ただしそれを保証しているのは書き手の記憶だけで、TypeScript は何も確かめていません。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)
    isActive: true,
  },
  'clx_user_123',
  updateUser,
  true,
);
```

このコードの問題点:

- `as UserEditFormValues` は「そういう型だと信じる」だけで、実際の値は検証していません
- `role: 'OWNER'` や `isActive: 'yes'` のような値でも、クライアント側では通ってしまいます
- フォーム項目が増えると型と実行時チェックのズレに気づきにくい
- `isAdmin` だけで分岐すると、管理者が自分を編集するときも `role` と `isActive` を送り、サーバーから `FORBIDDEN` が返ります

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)
import { z } from 'zod';
import { USER_ROLE, type UserRole } from
'@/lib/constant/roles';

const USER_ROLE_VALUES = ['USER', 'ADMIN'] as const;

const userEditSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  avatar:
z.string().url('有効なURLを入力してください').or(
z.literal('')),
  role: z.enum(USER_ROLE_VALUES),
  isActive: z.boolean(),
});

type UserEditFormValues = z.infer<typeof userEditSchema>;

type UpdateUserInput = {
  id: string;
  name: string;
  avatar?: string | null;
  role?: UserRole;
  isActive?: boolean;
};
```

`UpdateUserInput` の `avatar` は `null` を受け取れるようにします。空欄を `undefined` にすると既存画像を更新しない意味になるため、画像を削除するリクエストには `null` が必要です。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

type UpdateUserMutation = {
  mutate: (input: UpdateUserInput) => void;
};
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

After では型を用意する順番が逆になっています。先に `userEditSchema` を書き、そこから `z.infer` で型を取り出します。名前・アバター・ロール・アクティブの条件が置いてある場所が1か所だけになるので項目を足すときに直すのもスキーマだけです。Step 7 でフォームに渡したのと同じスキーマを、ここでは送信前の検査にも使い回しています。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

export function submitUserEditForm(
  rawValues: unknown,
  userId: string,
  updateUser: UpdateUserMutation,
  canManageAccount: boolean,
) {
  const values: UserEditFormValues = userEditSchema.parse(
    rawValues);

  updateUser.mutate({
    id: userId,
    name: values.name,
    avatar: values.avatar || null,
    ...(canManageAccount
      ? {
        role: values.role,
        isActive: values.isActive,
      }
      : {}),
  });
}

submitUserEditForm(
  {
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

`as` が `userEditSchema.parse(rawValues)` に置き換わりました。`parse` は値を1項目ずつスキーマと照らし合わせて合っていれば型の付いた値を返し、合っていないければその場で例外を投げます。`role` に `'OWNER'` が紛れ込んでいればサーバーへ送る前にここで止まります。左辺に `UserEditFormValues` と型を書いても `as` のような信じるだけの宣言にはなりません。中身が確かめられた後の値だからです。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

    name: 'Taro',
    avatar: '',
    role: USER_ROLE.ADMIN,
    isActive: true,
  },
  'clx_user_123',
  updateUser,
  true,
);
```

このコードの強み:

- zod が実行時にも値を検証するのでフォーム外から変な値が来ても止められます
- `z.infer` で TypeScript の型をスキーマから作るため型とバリデーションがズレにくい
- 管理者が他人を編集するときだけ `role` / `isActive` を送るルールと、入力値の安全性を分けて読める

覚えておきたいエッセンス

`as` は型チェックを黙らせる道具であって値を守る道具ではありません。ユーザー入力や API 境界では `zod` で実際の値を検証します。

完成コード全体

今日は5つのファイルに触りました。断片を貼り重ねる作業が11個の Step にまたがったので途中でどこへ貼ったか分からなくなった場合は以下のコードと手元のファイルを見比べてください。`user.ts` には Day 24 と Day 25 の処理を含め、Day 29 終了時点のファイル全体を載せます。他の4ファイルも全文です。

ファイル	役割	対応する Step
<code>src/server/api/routers/user.ts</code>	詳細を返す 入口と、更新を受け取る 出口	Step 0
<code>src/app/user/[id]/page.tsx</code>	詳細ページの存在確認と404	Step 2
<code>src/app/user/[id]/user-detail-client.tsx</code>	詳細画面の表示本体	Step 3 から Step 6
<code>src/app/user/[id]/edit/page.tsx</code>	編集ページの存在確認と404	Step 7

ファイル	役割	対応する Step
<code>src/app/user/[id]/edit/user-edit-client.tsx</code>	編集フォーム の本体	Step 7 か ら Step 10

`src/server/api/routers/user.ts`

これまでに作った手続きへ、今日の `getById` と `update` を加えた最終形です。スキーマとルーターの一部だけを貼り替えると宣言順や閉じ括弧を崩しやすいため、既存内容をファイル全体で置き換えてください。

Day 29: ユーザー詳細・編集ページを作ろう

```
// filepath: src/server/api/routers/user.ts

import { Prisma } from '@prisma/client';

import { TRPCError } from '@trpc/server';

import bcrypt from 'bcryptjs';

import { z } from 'zod';

import { USER_ROLE } from '@lib/constant/roles';

import { TASK_STATUS } from '@lib/constant/status';

import { createPasswordSchema } from '@lib/password';

import { prisma } from '@lib/prisma';

import { createSession, type SessionUser } from
'@lib/session';

import { adminProcedure, createTRPCRouter,
protectedProcedure } from '../trpc';

import { USER_DETAIL_SELECT } from '../_helpers/select';

import { isUserEmailUniqueConstraintError } from
'../_helpers/user-email-conflict';

const userUpdateSchema = z
  .object({
    id: z.string().cuid(),
    name: z.string().min(1,
'名前を入力してください').optional(),
    avatar: z.string().url().optional().nullable(),
    role: z.nativeEnum(USER_ROLE).optional(),
    isActive: z.boolean().optional(),
  })
  .refine(
    ({ name, avatar, role, isActive }) =>
```

```

    name !== undefined || avatar !== undefined ||
    role !== undefined || isActive !== undefined,

    '更新する項目を1つ以上指定してください',

  );

const profileUpdateSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  email:
    z.string().email('有効なメールアドレスを入力してください'),
  avatar: z.string().url().optional().nullable(),
});

const changePasswordSchema = z.object({
  currentPassword: z.string().min(1,
    '現在のパスワードを入力してください'),
  newPassword:
    createPasswordSchema('新しいパスワードは8文字以上で入力してください'),
});

const POSTGRES_INTEGER_MAX = 2_147_483_647;

async function tryReissueSession(path: string, user:
  SessionUser): Promise<boolean> {
  try {
    await createSession(user);
    return true;
  } catch {
    console.error('[auth] session reissue failed', {
      event: 'auth.session_reissue_failed',

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    path,

    userId: user.id,
  });

  return false;
}
}

export const userRouter = createTRPCRouter({

// 共通処理がDBの最新ロールで管理者判定を済ませるため、ここでは再確認しません
getAll: adminProcedure

  .input(
    z
      .object({
        isActive: z.boolean().optional(),
        role: z.nativeEnum(USER_ROLE).optional(),
      })
      .optional(),
  )

  .query(async ({ input }) => {

    const where: Prisma.UserWhereInput = {};

    if (input?.isActive !== undefined) {
      where.isActive = input.isActive;
    }

    if (input?.role) {
      where.role = input.role;
    }
  })
});
```

```

return await prisma.user.findMany({
  where,
  select: {
    ...USER_DETAIL_SELECT,
    createdAt: true,
    updatedAt: true,
  },
  orderBy: { createdAt: 'desc' },
});
}),

getById: protectedProcedure
  .input(z.object({ id: z.string().cuid() }))
  .query(async ({ ctx, input }) => {

// 本人またはADMINのみ他ユーザーの詳細情報にアクセス可能

    if (ctx.session.userId !== input.id &&
ctx.session.role !== USER_ROLE.ADMIN) {

      throw new TRPCError({
        code: 'FORBIDDEN',
        message: 'この操作を行う権限がありません',
      });
    }

    const user = await prisma.user.findUnique({
      where: { id: input.id },
      select: {
        ...USER_DETAIL_SELECT,

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    createdAt: true,

    updatedAt: true,

    projects: {

      include: {

        project: {

          select: {

            id: true,

            name: true,

            color: true,

          },

        },

      },

    },

    assignedTasks: {

      select: {

        id: true,

        title: true,

        status: true,

        priority: true,

        dueDate: true,

      },

      where: {

        status: {

          notIn: [TASK_STATUS.DONE,

TASK_STATUS.CANCELLED],

        },

        project: {

          members: {

            some: { userId: ctx.session.userId },
```

```

        },
        },
        },
        orderBy: { dueDate: 'asc' },
    },
    },
});

if (!user) {
    throw new TRPCError({
        code: 'NOT_FOUND',
        message: 'ユーザーが見つかりません',
    });
}

return user;
}),

update: protectedProcedure.input(
    userUpdateSchema).mutation(async ({ ctx, input }) => {
    const { id, ...data } = input;

    if (id !== ctx.session.userId) {

// 他ユーザーを更新する場合はセッションのroleでADMIN判定

        if (ctx.session.role !== USER_ROLE.ADMIN) {

            throw new TRPCError({

                code: 'FORBIDDEN',

                message: '管理者権限が必要です',

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    });  
  }  
} else {  
  
  // 自分のプロフィール更新の場合、roleとisActiveは変更不可  
  if (data.role !== undefined || data.isActive !==  
undefined) {  
    throw new TRPCError({  
      code: 'FORBIDDEN',  
      message:  
'ロールとアクティブ状態は変更できません',  
    });  
  }  
}  
  
const updateData: Prisma.UserUpdateInput = {};  
  
if (data.name !== undefined) {  
  updateData.name = data.name;  
}  
  
if (data.avatar !== undefined) {  
  updateData.avatar = data.avatar;  
}  
  
if (data.role !== undefined) {  
  updateData.role = data.role;  
}  
  
if (data.isActive !== undefined) {  
  updateData.isActive = data.isActive;  
  updateData.sessionVersion = { increment: 1 };  
}
```

```

try {
  return await prisma.user.update({
    where: {
      id,
      ...(data.isActive !== undefined ? {
sessionVersion: { lt: POSTGRES_INTEGER_MAX } } : {}),
    },
    data: updateData,
    select: {
      ...USER_DETAIL_SELECT,
      updatedAt: true,
    },
  });
} catch (err) {
  if (err instanceof
Prisma.PrismaClientKnownRequestError && err.code ===
'P2025') {
    if (data.isActive !== undefined) {
      throw new TRPCError({
        code: 'CONFLICT',
        message:
'セッションを更新できません。管理者にお問い合わせください',
      });
    }
    throw new TRPCError({
      code: 'NOT_FOUND',
      message: 'ユーザーが見つかりません',
    });
  }
}

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    }

    throw err;

  }

  }),

  updateProfile: protectedProcedure.input(
    profileUpdateSchema).mutation(async ({ ctx, input }) => {

    const userId = ctx.session.userId;

    if (input.email) {

      const existingUser = await prisma.user.findFirst({

        where: {

          email: input.email,

          id: { not: userId },

        },

      });

      if (existingUser) {

        throw new TRPCError({

          code: 'CONFLICT',

          message:

            'このメールアドレスは既に使用されています',

        });

      }

    }

    const updateData: Prisma.UserUpdateInput = {

      name: input.name,

      email: input.email,
```

```

};

if (input.avatar !== undefined) {
  updateData.avatar = input.avatar;
}

const updatedUser = await prisma.user
  .update({
    where: {
      id: userId,
      sessionVersion: ctx.session.version,
      isActive: true,
    },
    data: updateData,
    select: {
      ...USER_DETAIL_SELECT,
      updatedAt: true,
    },
  })
  .catch((err: unknown) => {
    if (err instanceof
Prisma.PrismaClientKnownRequestError && err.code ===
'P2025') {
      throw new TRPCError({
        code: 'UNAUTHORIZED',
        message:
'セッションが無効になりました。再度ログインしてください',
      });
    }

    if (isUserEmailUniqueConstraintError(err)) {

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    throw new TRPCError({
      code: 'CONFLICT',
      message:
        'このメールアドレスは既に使用されています',
    });
  }

  throw err;
});

const sessionReissued =
  input.email === ctx.session.email ||

  (await tryReissueSession('user.updateProfile', {
    id: updatedUser.id,
    email: updatedUser.email,
    role: updatedUser.role,
    version: ctx.session.version,
  }));

  return { ...updatedUser, success: true,
sessionReissued };
}),

changePassword: protectedProcedure
  .input(changePasswordSchema)
  .mutation(async ({ ctx, input }) => {
    const userId = ctx.session.userId;

    const user = await prisma.user.findUnique({
      where: { id: userId },
```

```
        select: { password: true, isActive: true,
sessionVersion: true },

    });

    if (!user) {

        throw new TRPCError({

            code: 'UNAUTHORIZED',

            message:
'セッションが無効になりました。再度ログインしてください',

        });
    }

    if (!user.isActive) {

        throw new TRPCError({

            code: 'FORBIDDEN',

            message: 'このアカウントは無効化されています',

        });
    }

    if (user.sessionVersion !== ctx.session.version) {

        throw new TRPCError({

            code: 'UNAUTHORIZED',

            message:
'セッションが無効になりました。再度ログインしてください',

        });
    }

    if (!user.password) {

        throw new TRPCError({
```

Day 29: ユーザー詳細・編集ページを作ろう

```
      code: 'BAD_REQUEST',

      message: 'パスワードが設定されていません',

    });
  }

  const isValidPassword = await bcrypt.compare(
    input.currentPassword, user.password);

  if (!isValidPassword) {
    throw new TRPCError({
      code: 'UNAUTHORIZED',

      message: '現在のパスワードが正しくありません',

    });
  }

  if (ctx.session.version === POSTGRES_INTEGER_MAX) {
    throw new TRPCError({
      code: 'CONFLICT',

      message:
        'セッションを更新できません。管理者にお問い合わせください',

    });
  }

  const hashedPassword = await bcrypt.hash(
    input.newPassword, 10);

  const updated = await prisma.user.updateMany({
    where: {
      id: userId,
```

```
    sessionVersion: ctx.session.version,

    password: user.password,

    isActive: true,
  },

  data: {

    password: hashedPassword,

    sessionVersion: { increment: 1 },
  },
});

if (updated.count !== 1) {

  throw new TRPCError({

    code: 'UNAUTHORIZED',

    message:

      'セッションが無効になりました。再度ログインしてください',

  });
}

const sessionReissued = await
tryReissueSession('user.changePassword', {

  id: userId,

  email: ctx.session.email,

  role: ctx.session.role,

  version: ctx.session.version + 1,
});

return {

  success: true,

  sessionReissued,
```

Day 29: ユーザー詳細・編集ページを作ろう

```
      message: 'パスワードを変更しました',  
    };  
  },  
});
```

`getId` は本人または管理者かを対象検索より先に判定し、権限のある人だけユーザー情報を取得します。`update` は他人の編集を管理者に限り、本人編集では `role` と `isActive` を拒否します。画面側の表示制御だけに頼らず、データを書き換える直前にも同じ境界を置いています。

`src/app/user/[id]/page.tsx`

詳細ページの入口は認証・認可・未発見を描画前に分けます。エラーごとの分岐を途中で欠かさないう、既存内容をファイル全体で置き換えてください。

```
// filepath: src/app/user/[id]/page.tsx  
import { getTRPCErrorFromUnknown } from '@trpc/server';  
import { notFound, redirect } from 'next/navigation';  
import { AppLayout } from '@component/layout/app-layout';  
import { Card, CardContent } from '@component/ui/card';  
import { trpc } from '@trpc/server';  
import { UserDetailsClient } from './user-detail-client';  
  
interface UserDetailsPageProps {  
  params: Promise<{  
    id: string;  
  }>;  
}
```

```

export default async function UserDetailPage({ params }:
UserDetailPageProps) {
  const { id } = await params;

  try {
    await trpc.user.getById({ id });
  } catch (error) {
    const trpcError = getTRPCErrorFromUnknown(error);
    if (trpcError.code === 'UNAUTHORIZED')
      redirect('/login');
    if (trpcError.code === 'NOT_FOUND') notFound();
    if (trpcError.code === 'FORBIDDEN') {
      return (
        <AppLayout>
          <div className="container mx-auto max-w-6xl
mt-8">
            <Card>
              <CardContent className="pt-6">
                <h1 className="text-2xl font-bold mb-2">
アクセス権限がありません</h1>
                <p className="text-muted-foreground">
このユーザーを見る権限がありません</p>
              </CardContent>
            </Card>
          </div>
        </AppLayout>
      );
    }
    throw error;
  }
}

```

```
}  
  
return <UserDetailClient userId={id} />;  
}
```

UNAUTHORIZED はログイン画面、**NOT_FOUND** は404、**FORBIDDEN** は拒否画面へ分けます。分類できない失敗は投げ直し、Day 26の **error.tsx** に処理を任せます。

src/app/user/[id]/user-detail-client.tsx

このファイルは取得状態の分岐と詳細画面を1つの部品として完成させます。途中だけを置き換えると、query の状態変数と表示側の分岐が食い違います。ここではファイル全体をコピーし、既存内容を置き換えてください。

```
// filepath: src/app/user/[id]/user-detail-client.tsx  
  
'use client';  
  
import { format } from 'date-fns';  
import { ja } from 'date-fns/locale';  
import { ArrowLeft, Calendar, Mail, Pencil } from  
'lucide-react';  
import Link from 'next/link';  
import { useRouter } from 'next/navigation';  
import { AppLayout } from '@component/layout/app-layout';  
import { StatusBadge } from  
'@component/task/status-badge';  
import { Avatar, AvatarFallback, AvatarImage } from  
'@component/ui/avatar';
```

```
import { Badge } from '@component/ui/badge';
import { Button } from '@component/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from
  '@component/ui/card';
import { PageLoadingSpinner } from
  '@component/ui/loading-spinner';
import { Separator } from '@component/ui/separator';
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from '@component/ui/table';
import { ActiveStatusBadge, UserRoleBadge } from
  '@component/ui/user-badges';
import { getPriorityBadgeVariant } from
  '@lib/badge-variant';
import { TASK_PRIORITY_LABELS } from
  '@lib/constant/priority';
import { USER_ROLE } from '@lib/constant/roles';
import { formatDateOnly } from '@lib/date';
import { httpStatusOf, isAuthError, isForbiddenError,
  shouldRetryQuery } from '@lib/query-error';
import { api } from '@trpc/react';

const shouldRetryUserQuery = (failureCount: number,
  error: unknown) =>
```

Day 29: ユーザー詳細・編集ページを作ろう

```
    httpStatusOf(error) !== 404 && shouldRetryQuery(
failureCount, error);

interface UserDetailsClientProps {
  userId: string;
}

export function UserDetailsClient({ userId }:
UserDetailsClientProps) {
  const router = useRouter();

  const {
    data: currentUser,
    isLoading: isCurrentUserLoading,
    isError: isCurrentUserError,
    isFetching: isCurrentUserFetching,
    error: currentUserError,
    refetch: refetchCurrentUser,
  } = api.auth.getCurrentUser.useQuery(undefined, {
retry: shouldRetryQuery });

  const {
    data: user,
    isLoading: isUserLoading,
    isError: isUserError,
    isFetching: isUserFetching,
    error: userError,
    refetch: refetchUser,
  } = api.user.getById.useQuery(
```

```

    { id: userId },
    { enabled: userId.length > 0, retry:
shouldRetryUserQuery },
  );

  const queryErrors = [
    isCurrentUserError ? currentUserError : null,
    isUserError ? userError : null,
  ];

  const authFailed = queryErrors.some(isAuthError);
  const forbidden = queryErrors.some(isForbiddenError);
  const notFound = isUserError && httpStatusOf(userError)
=== 404;

  const hasFetchError = isCurrentUserError || isUserError;
  const hasRequiredData =
    (!isCurrentUserError || currentUser != null) &&
(!isUserError || user != null);

  const requiredLoading = isCurrentUserLoading ||
isUserLoading;

  const requiredFetching = isCurrentUserFetching ||
isUserFetching;

  const refetchRequiredData = () => {
    void refetchCurrentUser();
    void refetchUser();
  };

  if (requiredLoading && !authFailed && !forbidden
&& !notFound) {

```

Day 29: ユーザー詳細・編集ページを作ろう

```
return (  
  <AppLayout>  
    <PageLoadingSpinner />  
  </AppLayout>  
);  
}  
  
if (  
  authFailed ||  
  forbidden ||  
  notFound ||  
  (hasFetchError && !hasRequiredData) ||  
  !currentUser ||  
  !user  
) {  
  return (  
    <AppLayout>  
      <div className="flex flex-col items-center  
justify-center py-24 text-center">  
        <p className="mb-2 text-base font-semibold  
text-foreground">  
          {authFailed  
            ? 'ログインの有効期限が切れました'  
            : forbidden  
            ? 'このユーザーを見る権限がありません'  
            : notFound  
            ? 'ユーザーが見つかりません'  
            : 'ユーザー情報を取得できませんでした'}  
        </p>  
      </div>  
    </AppLayout>  
  );  
}
```

```

    <p className="mb-6 text-sm
text-muted-foreground">
      {authFailed
        ? 'もう一度ログインしてください。'
        : forbidden
        ? '管理者または本人のみ表示できます。'
        : notFound
        ?
'削除されたか、URLが正しくない可能性があります。'
        :
'通信状況を確認して、再読み込みしてください。'}
    </p>

    <Button
      type="button"
      onClick={() => {
        if (authFailed) {
          router.push('/login');
          return;
        }
        if (forbidden) {
          router.push('/dashboard');
          return;
        }
        if (notFound) {
          router.push(currentUser?.role ===
USER_ROLE.ADMIN ? '/user' : '/dashboard');
          return;
        }
      }}
      refetchRequiredData();

```

Day 29: ユーザー詳細・編集ページを作ろう

```
    }}  
  
    disabled={requiredFetching}  
  
    >  
      {authFailed  
        ? 'ログイン画面へ'  
        : forbidden  
        ? 'ダッシュボードへ'  
        : notFound  
        ? currentUser?.role === USER_ROLE.ADMIN  
        ? 'ユーザー一覧へ'  
        : 'ダッシュボードへ'  
        : '再読み込み'}  
      </Button>  
    </div>  
  </AppLayout>  
);  
}
```

```
const isAdmin = currentUser?.role === USER_ROLE.ADMIN;  
const isOwnProfile = currentUser?.id === user.id;
```

```
return (  
  <AppLayout>  
    <div className="container mx-auto max-w-6xl py-8">  
      {hasFetchError && (  
        <div  
          role="alert"  
          className="mb-4 flex items-center  
            justify-between gap-4 rounded-lg border
```

```

border-amber-300/60 bg-amber-50 px-4 py-3 text-sm
text-amber-900 dark:border-amber-500/40
dark:bg-amber-950/40 dark:text-amber-200"

    >

    <span>
最新のユーザー情報を取得できませんでした。前回取得時の内容です。</span>

    <Button
      type="button"
      variant="outline"
      size="sm"
      onClick={refetchRequiredData}
      disabled={requiredFetching}
    >
      再試行
    </Button>
  </div>
)}

<Button
  variant="ghost"
  className="mb-4 pl-0 hover:bg-transparent
hover:text-primary"
  onClick={() => router.push(isAdmin ? '/user' :
'/profile')}
  >
    <ArrowLeft className="mr-2 h-4 w-4" />
    {isAdmin ? 'ユーザー一覧に戻る' :
'プロフィールに戻る'}
  </Button>

```

Day 29: ユーザー詳細・編集ページを作ろう

```
<div className="grid gap-6 md:grid-cols-12">

  <div className="md:col-span-4 space-y-6">

    <Card>

      <CardContent style={{ paddingTop: '2.5rem'
}}>

        <div className="text-center mb-6">

          <Avatar className="w-24 h-24 mx-auto
mb-4">

            {user.avatar && <AvatarImage src={
user.avatar} alt={user.name || ''} />}

            <AvatarFallback className="text-3xl">

              {user.name?.[0]?.toUpperCase()}

            </AvatarFallback>

          </Avatar>

          <h2 className="text-xl font-bold mb-2
break-words">{user.name || user.email}</h2>

          <div className="flex justify-center
gap-2 mb-4">

            <UserRoleBadge role={user.role} />

            <ActiveStatusBadge isActive={
user.isActive} />

          </div>

        </div>

      </div>

      <Separator className="my-4" />

      <div className="space-y-4 text-sm">

        <div className="flex items-center
gap-3">
```

```

        <Mail className="h-4 w-4
text-muted-foreground" />

        <div>

            <p className="font-medium
text-muted-foreground text-xs">メールアドレス</p>

            <p>{user.email}</p>

        </div>

    </div>

    <div className="flex items-center
gap-3">

        <Calendar className="h-4 w-4
text-muted-foreground" />

        <div>

            <p className="font-medium
text-muted-foreground text-xs">登録日</p>

            <p>

                {user.createdAt

                    ? format(new Date(
user.createdAt), 'yyyy年MM月dd日', {
                        locale: ja,
                    })
                    : '-'}

            </p>

        </div>

    </div>

    <div className="flex items-center
gap-3">

        <Calendar className="h-4 w-4
text-muted-foreground" />

```

Day 29: ユーザー詳細・編集ページを作ろう

```
        <div>
            <p className="font-medium
text-muted-foreground text-xs">最終更新日</p>

            <p>
                {user.updatedAt
                    ? format(new Date(
user.updatedAt), 'yyyy年MM月dd日', {
                        locale: ja,
                    })
                    : '-'}
            </p>
        </div>
    </div>
</div>

    {(isAdmin || isOwnProfile) && (
        <>
            <Separator className="my-4" />
            <Button className="w-full"
onClick={() => router.push(`/user/${user.id}/edit`)}>
                <Pencil className="mr-2 h-4 w-4"
/> 編集
            </Button>
        </>
    )}
</CardContent>
</Card>
</div>
```

```

<div className="md:col-span-8 space-y-6">

  <Card>

    <CardHeader>

      <CardTitle className="text-lg">
参加プロジェクト</CardTitle>

    </CardHeader>

    <CardContent>

      {user.projects && user.projects.length >
0 ? (

        <div className="flex flex-wrap gap-2">

          {user.projects.map((member) => (

            <Link

              key={member.id}

              href={` /project?projectId=${
member.project.id}`}

              className={

                'inline-flex items-center gap-2 rounded-md border px-3 py-1 '
+

                'text-sm text-foreground hover:bg-muted'

              }

            >

              <span

                aria-hidden="true"

                className="h-2.5 w-2.5

rounded-full"

                style={{ backgroundColor:
member.project.color }}

```

Day 29: ユーザー詳細・編集ページを作ろう

```
        />
        {member.project.name}
      </Link>
    )}}
  </div>
) : (
  <p className="text-muted-foreground
text-sm">
    参加しているプロジェクトはありません
  </p>
)}
</CardContent>
</Card>

<Card>
  <CardHeader>
    <CardTitle className="text-lg">
      担当中のタスク</CardTitle>
    </CardHeader>
    <CardContent className="p-0">
      {user.assignedTasks &&
      user.assignedTasks.length > 0 ? (
        <Table>
          <TableHeader>
            <TableRow>
              <TableHead>タイトル</TableHead>
              <TableHead>ステータス</TableHead>
              <TableHead>優先度</TableHead>
              <TableHead>期限</TableHead>
```

```

        </TableRow>
    </TableHeader>
    <TableBody>
        {user.assignedTasks.map((task) => (
            <TableRow key={task.id}>
                <TableCell
                    className="font-medium">
                        <Link
                            href={` /task?taskId=${
                                task.id}`}
                            className="text-primary
                                underline-offset-4 hover:underline"
                        >
                            {task.title}
                        </Link>
                    </TableCell>
                    <TableCell>
                        <StatusBadge status={
                            task.status} />
                    </TableCell>
                    <TableCell>
                        <Badge variant={
                            getPriorityBadgeVariant(task.priority)}>
                            {TASK_PRIORITY_LABELS[
                                task.priority]}
                        </Badge>
                    </TableCell>
                    <TableCell>{task.dueDate ?
                        formatDateOnly(task.dueDate) : '-'}</TableCell>

```

Day 29: ユーザー詳細・編集ページを作ろう

```
        </TableRow>
      )})
    </TableBody>
  </Table>
) : (
  <div className="p-6
text-muted-foreground text-sm">
    担当中のタスクはありません
  </div>
)}
</CardContent>
</Card>
</div>
</div>
</div>
</AppLayout>
);
}
```

`getCurrentUser` と `getById` のどちらが失敗しても、401・403・404・一時的な取得失敗を別々に扱います。401・403ではキャッシュ済みの詳細を隠し、500相当の再取得失敗では前回の内容を残して再試行できるようにしています。正常時は権限を確認したユーザーだけが詳細と編集ボタンへ進めます。

`src/app/user/[id]/edit/page.tsx`

編集URLは詳細ページを経由せず直接開けるため、編集側にも同じ server wrapper が必要です。既存内容をファイル全体で置き換えてください。

```
// filepath: src/app/user/[id]/edit/page.tsx
import { getTRPCErrorFromUnknown } from '@trpc/server';
import { notFound, redirect } from 'next/navigation';
import { AppLayout } from '@component/layout/app-layout';
import { Card, CardContent } from '@component/ui/card';
import { trpc } from '@trpc/server';
import { UserEditClient } from './user-edit-client';

interface UserEditPageProps {
  params: Promise<{
    id: string;
  }>;
}

export default async function UserEditPage({ params }:
UserEditPageProps) {
  const { id } = await params;

  try {
    await trpc.user.getById({ id });
  } catch (error) {
    const trpcError = getTRPCErrorFromUnknown(error);
    if (trpcError.code === 'UNAUTHORIZED')
      redirect('/login');
    if (trpcError.code === 'NOT_FOUND') notFound();
    if (trpcError.code === 'FORBIDDEN') {
      return (
        <AppLayout>
          <div className="container mx-auto max-w-md
```

Day 29: ユーザー詳細・編集ページを作ろう

```
mt-8">

    <Card>
      <CardContent className="pt-6">
        <h1 className="text-xl font-bold mb-2">
アクセス権がありません</h1>
        <p className="text-muted-foreground">
管理者または本人のみユーザー編集が可能です</p>
      </CardContent>
    </Card>
  </div>
</AppLayout>
);
}
throw error;
}

return <UserEditClient userId={id} />;
}
```

詳細側と同じ `getById` を使うことで、一般ユーザーが他人の实在IDと存在しないIDを開いた場合に同じ拒否画面を返します。成功したときだけ編集用 client へ ID を渡します。

`src/app/user/[id]/edit/user-edit-client.tsx`

このファイルはフォームの状態、取得失敗時の行き先、保存処理をまとめて完成させます。分割した断片を貼り継ぐとフックの順序や閉じタグを崩しやすいため、既存内容をファイル全体で置き換えてください。

```
// filepath: src/app/user/[id]/edit/user-edit-client.tsx

'use client';

import { zodResolver } from '@hookform/resolvers/zod';
import { AlertCircle, ArrowLeft } from 'lucide-react';
import { useRouter } from 'next/navigation';
import { useEffect, useRef } from 'react';
import { useForm } from 'react-hook-form';
import toast from 'react-hot-toast';
import { z } from 'zod';

import { AppLayout } from '@/component/layout/app-layout';
import { Alert, AlertDescription, AlertTitle } from
 '@/component/ui/alert';
import { Avatar, AvatarFallback, AvatarImage } from
 '@/component/ui/avatar';
import { Button } from '@/component/ui/button';
import { Card, CardContent, CardHeader, CardTitle } from
 '@/component/ui/card';
import { Checkbox } from '@/component/ui/checkbox';
import { Input } from '@/component/ui/input';
import { Label } from '@/component/ui/label';
import { PageLoadingSpinner } from
 '@/component/ui/loading-spinner';
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
```

Day 29: ユーザー詳細・編集ページを作ろう

```
} from '@component/ui/select';

import { isUserRole, USER_ROLE, USER_ROLE_LABELS } from
  '@lib/constant/roles';

import { httpStatusOf, isAuthError, isForbiddenError,
  shouldRetryQuery } from '@lib/query-error';

import { normalizeAvatarValue } from '@lib/utils';

import { api } from '@trpc/react';

const shouldRetryUserQuery = (failureCount: number,
  error: unknown) =>
  httpStatusOf(error) !== 404 && shouldRetryQuery(
    failureCount, error);

const USER_ROLE_VALUES = ['USER', 'ADMIN'] as const;

const userEditSchema = z.object({
  name: z.string().min(1, '名前を入力してください'),
  avatar:
    z.string().url('有効なURLを入力してください').or(
      z.literal('')),
  role: z.enum(USER_ROLE_VALUES),
  isActive: z.boolean(),
});

type UserEditFormValues = z.infer<typeof userEditSchema>;

interface UserEditClientProps {
  userId: string;
}
```

```

export function UserEditClient({ userId }:
UserEditClientProps) {
  const router = useRouter();

  const form = useForm<UserEditFormValues>({
    resolver: zodResolver(userEditSchema),
    defaultValues: {
      name: '',
      avatar: '',
      role: USER_ROLE.USER,
      isActive: true,
    },
  });

  const { dirtyFields } = form.formState;
  const dirtyFieldsRef = useRef(dirtyFields);
  dirtyFieldsRef.current = dirtyFields;

  const hydratedUserIdRef = useRef<string | null>(null);

  const {
    data: currentUser,
    isLoading: isCurrentUserLoading,
    isError: isCurrentUserError,
    isFetching: isCurrentUserFetching,
    error: currentUserError,
    refetch: refetchCurrentUser,
  } = api.auth.getCurrentUser.useQuery(undefined, {
    retry: shouldRetryQuery });

  const isAdmin = currentUser?.role === USER_ROLE.ADMIN;
  const isOwnProfile = currentUser?.id === userId;

```

Day 29: ユーザー詳細・編集ページを作ろう

```
const canEditUser = isAdmin || isOwnProfile;

const canManageAccount = isAdmin && !isOwnProfile;

const {
  data: user,
  isLoading: isUserLoading,
  isError: isUserError,
  isFetching: isUserFetching,
  error: userError,
  refetch: refetchUser,
} = api.user.getById.useQuery(
  { id: userId },
  {
    enabled: !!currentUser && canEditUser &&
    userId.length > 0,
    retry: shouldRetryUserQuery,
  },
);

const submitContextRef = useRef({ targetUserId: userId,
user, canEditUser, canManageAccount });

submitContextRef.current = { targetUserId: userId,
user, canEditUser, canManageAccount };

const utils = api.useUtils();

const updateUser = api.user.update.useMutation({
  onSuccess: async () => {
    await Promise.allSettled([
      utils.user.getById.invalidate({ id: userId }),
      utils.user.getAll.invalidate(),
    ]);
  },
});
```

```

    utils.auth.getCurrentUser.invalidate(),

    utils.auth.getSession.invalidate(),

  });

  toast.success('ユーザー情報を更新しました');

  router.push(`/user/${userId}`);

  router.refresh();

},

onError: (error) => {

  toast.error(error.message ??

'ユーザー情報の更新に失敗しました');

},

});

useEffect(() => {

  if (user) {

    const sameUser = hydratedUserIdRef.current ===

user.id;

    form.reset(

      {

        name: user.name ?? '',

        avatar: user.avatar ?? '',

        role: user.role,

        isActive: user.isActive,

      },

      sameUser && Object.keys(

dirtyFieldsRef.current).length > 0

        ? { keepDirtyValues: true }

        : undefined,

    );

```

Day 29: ユーザー詳細・編集ページを作ろう

```
hydratedUserIdRef.current = user.id;
}
}, [user, form]);

const handleSubmit = (values: UserEditFormValues) => {
  const submitContext = submitContextRef.current;
  if (
    !submitContext.user ||
    !submitContext.canEditUser ||
    submitContext.targetUserId !== userId ||
    submitContext.user.id !== userId
  )
    return;
  updateUser.mutate({
    id: userId,
    name: values.name,
    avatar: normalizeAvatarValue(values.avatar),

    // 同じ値を送り直すと別の管理者による更新を古いフォームで巻き戻すためです。
    ...(submitContext.canManageAccount &&
    values.role !== submitContext.user.role
      ? { role: values.role }
      : {}),
    ...(submitContext.canManageAccount &&
    values.isActive !== submitContext.user.isActive
      ? { isActive: values.isActive }
      : {}),
  });
};
```

```

const queryErrors = [
  isCurrentUserError ? currentUserError : null,
  isUserError ? userError : null,
];

const authFailed = queryErrors.some(isAuthError);
const forbidden = queryErrors.some(isForbiddenError);
const notFound = isUserError && httpStatusOf(userError)
=== 404;

const hasFetchError = isCurrentUserError || isUserError;
const hasRequiredData =
  (!isCurrentUserError || currentUser != null) &&
(!isUserError || user != null);

const requiredLoading = isCurrentUserLoading || (
canEditUser && isUserLoading);

const requiredFetching = isCurrentUserFetching ||
isUserFetching;

const refetchRequiredData = () => {
  void refetchCurrentUser();
  if (canEditUser) void refetchUser();
};

if (requiredLoading && !authFailed && !forbidden
&& !notFound) {
  return (
    <AppLayout>
      <PageLoadingSpinner />
    </AppLayout>
  )
}

```

Day 29: ユーザー詳細・編集ページを作ろう

```
);  
  
}  
  
if (authFailed || forbidden || notFound || (  
hasFetchError && !hasRequiredData) || !currentUser) {  
  return (  
    <AppLayout>  
      <div className="flex flex-col items-center  
justify-center py-24 text-center">  
        <p className="mb-2 text-base font-semibold  
text-foreground">  
          {authFailed  
            ? 'ログインの有効期限が切れました'  
            : forbidden  
            ? 'このユーザーを編集する権限がありません'  
            : notFound  
            ? 'ユーザーが見つかりません'  
            : 'ユーザー情報を取得できませんでした'}  
        </p>  
        <p className="mb-6 text-sm  
text-muted-foreground">  
          {authFailed  
            ? 'もう一度ログインしてください.'  
            : forbidden  
            ? '管理者または本人のみ編集できます.'  
            : notFound  
            ?  
            '削除されたか、URLが正しくない可能性があります.'  
            :  
          }  
        </p>  
      </div>  
    </AppLayout>  
  );  
}
```

```

'通信状況を確認して、再読み込みしてください。'}

</p>

<Button

  type="button"

  onClick={() => {

    if (authFailed) {

      router.push('/login');

      return;

    }

    if (forbidden) {

      router.push('/dashboard');

      return;

    }

    if (notFound) {

      router.push(isAdmin ? '/user' :

'/dashboard');

      return;

    }

    refetchRequiredData();

  }}

  disabled={requiredFetching}

>

  {authFailed

    ? 'ログイン画面へ'

    : forbidden

    ? 'ダッシュボードへ'

    : notFound

    ? isAdmin

    ? 'ユーザー一覧へ'

```

Day 29: ユーザー詳細・編集ページを作ろう

```
                : 'ダッシュボードへ'

                : '再読み込み'}

        </Button>

    </div>

</AppLayout>

);

}

if (!canEditUser) {

    return (

        <AppLayout>

            <div className="container mx-auto max-w-md mt-8">

                <Card>

                    <CardContent className="pt-6">

                        <h1 className="text-xl font-bold mb-2">

アクセス権がありません</h1>

                        <p className="text-muted-foreground">

管理者または本人のみユーザー編集が可能です</p>

                    </CardContent>

                </Card>

            </div>

        </AppLayout>

    );

}

if (!user) {

    return (

        <AppLayout>

            <div className="flex flex-col items-center
```

```

justify-center py-24 text-center">

    <p className="mb-2 text-base font-semibold
text-foreground">

        ユーザー情報を取得できませんでした

    </p>

    <p className="mb-6 text-sm
text-muted-foreground">

        通信状況を確認して、再読み込みしてください。

    </p>

    <Button onClick={refetchRequiredData} disabled={
requiredFetching}>

        再読み込み

    </Button>

</div>

</AppLayout>

);

}

return (

    <AppLayout>

        <div className="container mx-auto max-w-md mt-8
mb-8">

            {hasFetchError && (

                <div

                    role="alert"

                    className="mb-4 flex items-center
justify-between gap-4 rounded-lg border
border-amber-300/60 bg-amber-50 px-4 py-3 text-sm
text-amber-900 dark:border-amber-500/40

```

Day 29: ユーザー詳細・編集ページを作ろう

```
dark:bg-amber-950/40 dark:text-amber-200"

    >

    <span>

最新のユーザー情報を取得できませんでした。入力内容は保持されています。</span>

    <Button

      type="button"

      variant="outline"

      size="sm"

      onClick={refetchRequiredData}

      disabled={requiredFetching}

    >

      再試行

    </Button>

  </div>

)}

<Button

  variant="ghost"

  className="mb-4 pl-0 hover:bg-transparent

hover:text-primary"

  onClick={() => router.push(`/user/${userId}`)}

  >

    <ArrowLeft className="mr-2 h-4 w-4" />

    戻る

  </Button>

<Card>

  <CardHeader>

    <CardTitle>ユーザー編集</CardTitle>

  </CardHeader>
```

```

<CardContent>

  <form onSubmit={form.handleSubmit(
handleSubmit)} className="space-y-6">

    <div className="flex justify-center mb-6">

      <Avatar className="w-24 h-24">

        {form.watch('avatar') && <AvatarImage
src={form.watch('avatar')} alt="" />}

        <AvatarFallback className="text-2xl">

          {form.watch('name')?.[
0]?.toUpperCase()}

        </AvatarFallback>

      </Avatar>

    </div>

    <div className="space-y-2">

      <Label htmlFor="name">

        名前{' '}

        <span aria-hidden="true"
className="text-destructive">

          *

        </span>

      </Label>

      <Input

        id="name"

        aria-required="true"

        {...form.register('name')}

        disabled={updateUser.isPending}

      />

      {form.formState.errors.name && (

```

Day 29: ユーザー詳細・編集ページを作ろう

```
        <p className="text-sm
text-destructive">{form.formState.errors.name.message}</p>

      )}

    </div>

    <div className="space-y-2">

      <Label htmlFor="email">
メールアドレス</Label>

      <Input id="email" value={user.email}
disabled />

      <p className="text-xs
text-muted-foreground">メールアドレスは変更できません</p>

    </div>

    <div className="space-y-2">

      <Label htmlFor="avatar">
アバターURL(任意)</Label>

      <Input

        id="avatar"

        type="url"

        {...form.register('avatar')}

        disabled={updateUser.isPending}

        placeholder="https://example.com/avatar.png"

      />

    </div>

    {canManageAccount && (

      <>
```

```

<div className="space-y-2">

  <Label htmlFor="role">ロール</Label>

  <Select

    value={form.watch('role')}

    onChange={({value}) => {

      if (isUserRole(value)) {

        form.setValue('role', value, {
shouldDirty: true });

      }

    }}

    disabled={updateUser.isPending}

  >

    <SelectTrigger id="role">

      <SelectValue

placeholder="ロールを選択" />

      </SelectTrigger>

      <SelectContent>

        {Object.entries(
USER_ROLE_LABELS).map(([value, label]) => (

          <SelectItem key={value} value={
value}>

            {label}

          </SelectItem>

        ))}

      </SelectContent>

    </Select>

  </div>

  <div className="flex items-center

```

Day 29: ユーザー詳細・編集ページを作ろう

```
space-x-2">

      <Checkbox

        id="isActive"

        checked={form.watch('isActive')}

        onChangeChange={({checked}) =>
form.setValue('isActive', checked === true, {
shouldDirty: true }}}

        disabled={updateUser.isPending}

      />

      <Label htmlFor="isActive">
アクティブ</Label>

    </div>

  </>

)}

{updateUser.error && (

  <Alert variant="destructive">

    <AlertCircle className="h-4 w-4" />

    <AlertTitle>エラー</AlertTitle>

    <AlertDescription>{
updateUser.error.message}</AlertDescription>

  </Alert>

)}

  <div className="flex gap-2 pt-2">

    <Button type="submit" className="w-full"
disabled={updateUser.isPending}>

      {updateUser.isPending ? '更新中...' :
'更新'}
```

```

        </Button>

        <Button
            type="button"
            variant="outline"
            className="w-full"
            onClick={() => router.push(`/user/${
userId}`)}

            disabled={updateUser.isPending}
        >
            キャンセル
        </Button>
    </div>
</Form>

</CardContent>

</Card>

</div>

</AppLayout>

);
}

```

`canEditUser` は編集ページへ入れるかを決め、`canManageAccount` はロールとアクティブ状態を送れるかを決めます。`submitContextRef` は最新の対象ID、取得データ、権限を保存し、画面遷移前の送信処理が別の利用者へ値を送るのを防ぎます。保存時は空のアバターURLを `normalizeAvatarValue` で `null` に変えます。管理者が他人を編集する場合も、読み込んだ値から変えた `role` と `isActive` だけを送ります。

今日のまとめ

今日で管理者向けのユーザー管理機能が完成しました。詳細表示・編集・権限チェックまで実装し、あなたのタスク管理アプリは本格的なチーム管理ツールになりました。動的ルーティングでURLから情報を読み取り、権限に基づいてUIを切り替える方法を学びました。

- ☐ `[id]` の動的ルーティングで URL からユーザー ID を取り出せました
- ☐ `server wrapper` と `client component` の役割を分けて書けました
- ☐ 初回と別ユーザーでは全項目を初期化し、同じユーザーの再取得では入力途中の項目を保持できました
- ☐ 管理者と本人で編集できる項目が変わることを確かめました
- ☐ 保存後に詳細ページへ戻り、更新後の値が出ることを確かめました

つまずきポイント

ページが404になる

原因

`[id]` フォルダ名の角括弧が全角のためです。

解決方法

半角の `[id]` でフォルダを作り直してください。

`await params` の `id` が `undefined` になる

原因

フォルダ名の括弧内と `params` のキー名が一致していないためです。

解決方法

`[userId]` なら `params.userId`、`[id]` なら `params.id` を読んでください。

フォームの初期値が空になる

原因

Step 8 の `useEffect` で初回の `form.reset` が動いていないためです。

解決方法

`hydratedUserIdRef` と `dirtyFields` を用意し、Step 8 の `useEffect` が書けているか確認してください。別の利用者では全項目を初期化し、同じ利用者では `keepDirtyValues` を使います。

権限チェックが効かない

原因

開いているのが他人のページで、現在のデータベース上のロールが ADMIN でないためです。`USER_ROLE.ADMIN` の実値は `'ADMIN'` なので、文字列と定数のどちらで比較しても値が同じなら判定結果は変わりません。サーバーは保護された API を呼ぶたびに最新のロールをデータベースから読み、`user.getById` と `user.update` で「本人または ADMIN」かを判定します。

解決方法

Day 29: ユーザー詳細・編集ページを作ろう

まず URL のユーザー ID と `getCurrentUser` の `id` を比べ、本人のページか確認してください。他人のページなら、`getCurrentUser` の `role` が `'ADMIN'` である必要があります。ロールを変更した直後に画面の表示が古い場合は、ページを再読み込みして `getCurrentUser` を取得し直します。それでもサーバーが拒否する場合は、データベース上のそのユーザーの `role` と、開いている対象 ID を確認します。最新のロールは API ごとに読み直されるため、ロールの反映だけを理由に再ログインする必要はありません。

更新後に古いデータが表示される

原因

tRPC のキャッシュが残っているためです。

解決方法

`onSuccess` で `user/auth` を `invalidate` してから詳細へ戻ってください。

`FORBIDDEN` エラーが表示される

原因

本人更新で `role` ・ `isActive` を送信しているためです。

解決方法

`canManageAccount` が `true` で、読み込んだ値から変更した項目だけ送信してください。画面遷移直前の送信にも備えます。 `submitContextRef` の `targetUserId` と現在の URL の ID が一致することも確認します。

今日学んだ用語

用語	意味
動的ルーティング <code>[id]</code>	フォルダ名を変数にして任意の URL を 1つのページで受ける仕組み
<code>await params</code>	server component で URL のパラメータを受け取る書き方。Next.js 15 から非同期になった
<code>useEffect</code> の依存配列	指定した値が変わったときだけ処理を走らせるための指定
<code>zodResolver</code>	zod のスキーマを <code>useForm</code> のバリデーションにつなぐ関数
<code>form.reset</code>	フォームの値をまとめて入れ直す。サーバーのデータが届いたときに使う
早期リターン	読み込み中・未発見・権限なしを先に返し、最後に本体を描く書き方
<code>isPending</code>	mutation が処理中かどうか。2重送信を止めるのに使う
<code>md:grid-cols-12</code>	画面が広いときだけ12列のグリッドにする指定

全体のデータフロー振り返り

処理を3つに分けると、画面を開く前の判定、フォームへの反映、保存後の移動を追いやすくなります。エラーが出た場所も、この3つのどこかに絞って確認できます。

URLから表示するユーザーを決める

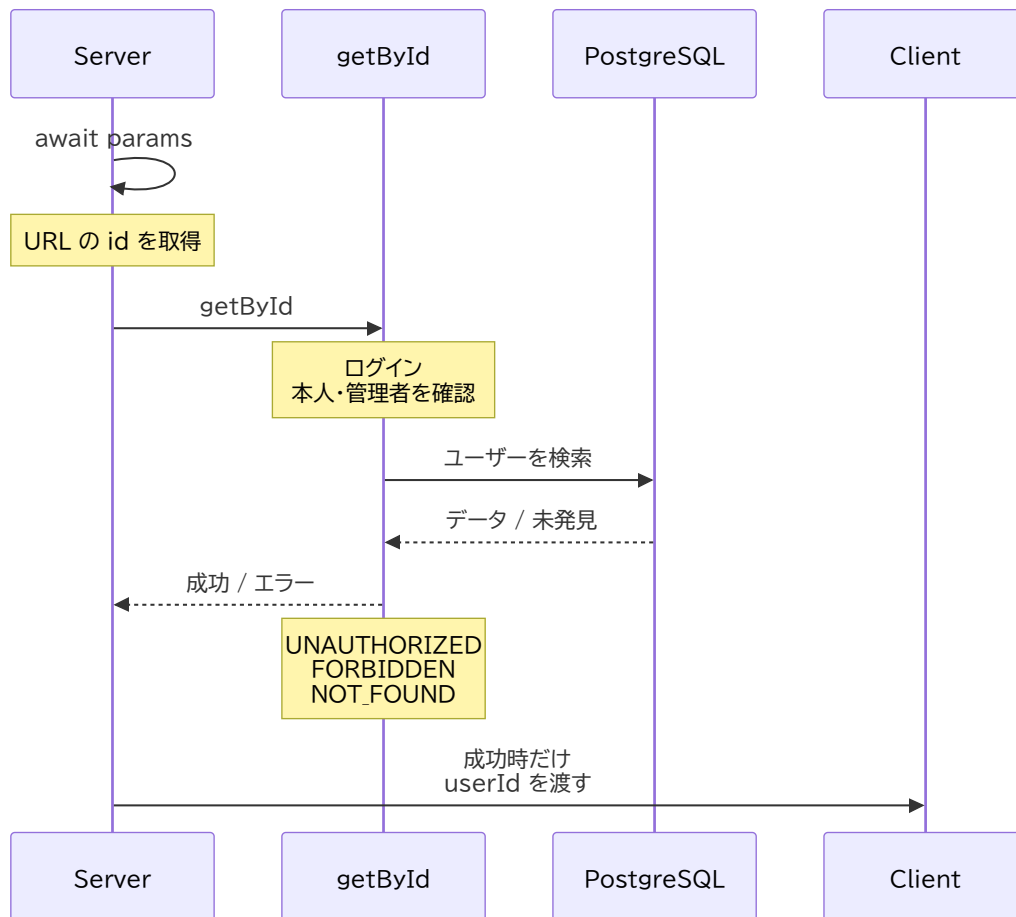


図 7: URLから表示するユーザーを決める

server wrapper は最初の `getById` で本人・管理者・未発見を分けます。成功した場合だけ `userId` を client component に渡すため、権限のない人には編集フォームを表示しません。

取得した値をフォームへ入れる

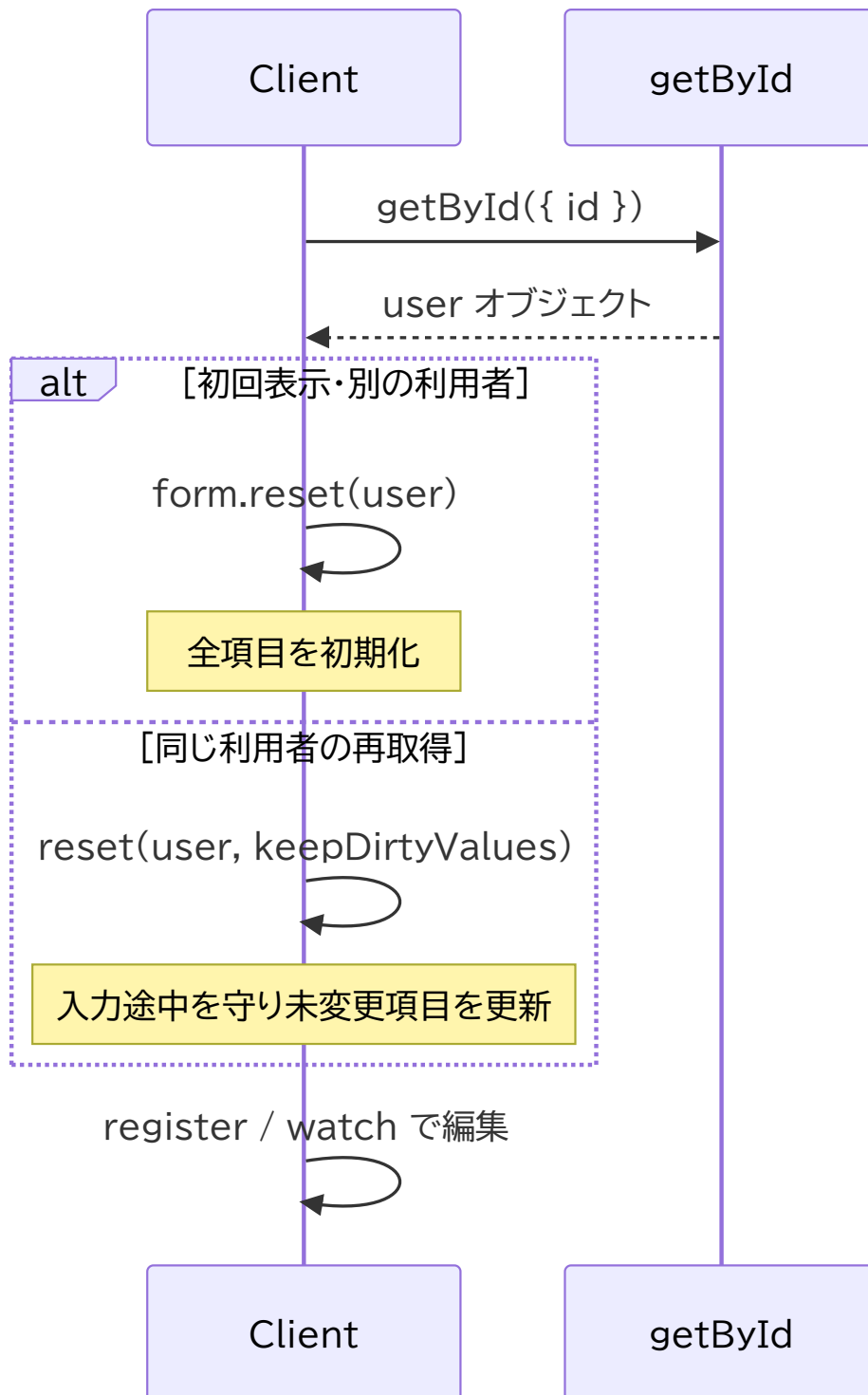


図8: 取得した値をフォームへ入れる

Day 29: ユーザー詳細・編集ページを作ろう

client component は表示用のデータを再取得します。初回と別の利用者では全項目を入れ直します。同じ利用者では入力途中の項目を残し、未変更項目だけを取得結果へ更新します。`register` と `watch` はその後の編集をフォームの値として扱います。

更新を許可して詳細ページへ戻る

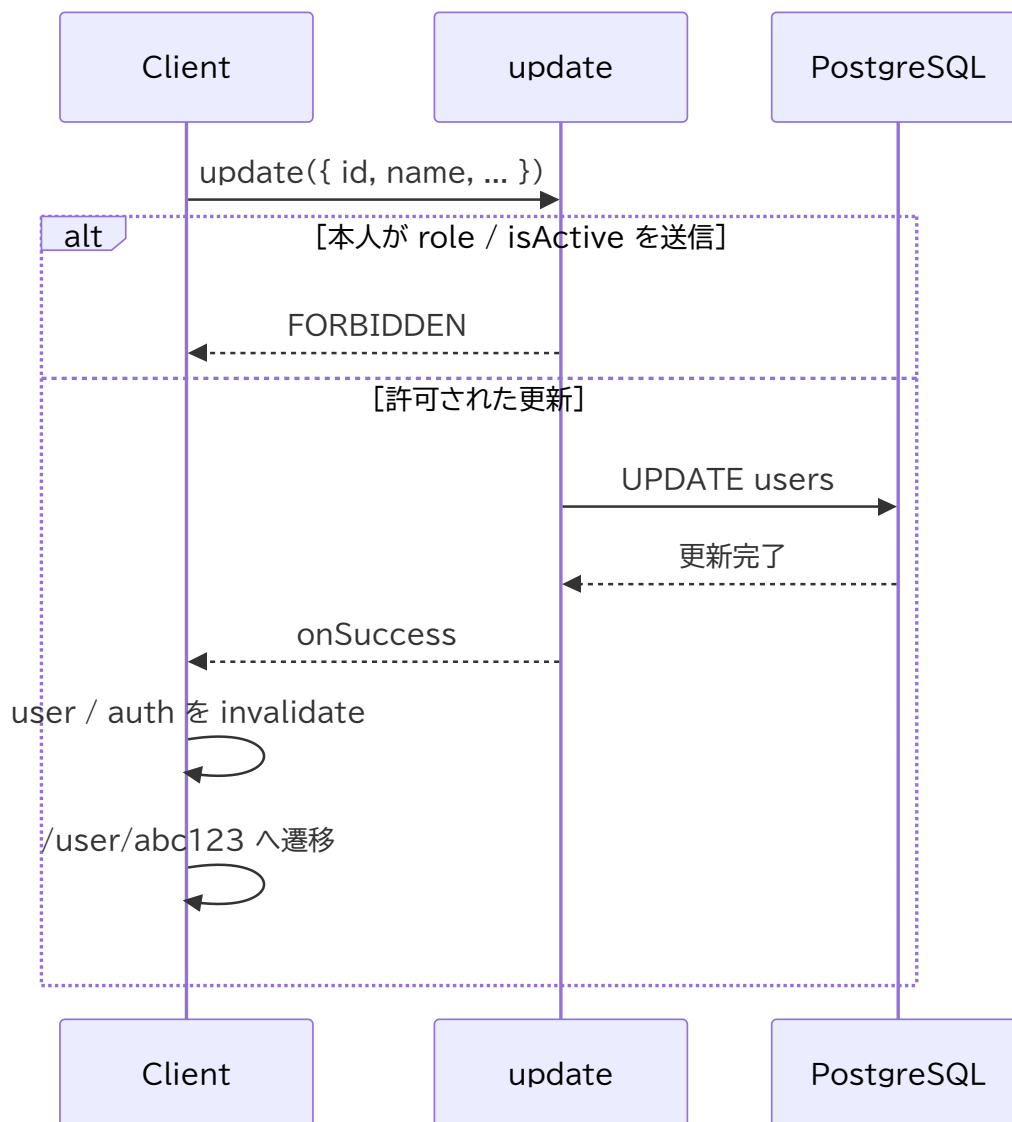


図 9: 更新を許可して詳細ページへ戻る

本人は名前とアバターを更新できます。管理者は `role` と `isActive` も更新できます。成功後は `onSuccess` で `user/auth` のキャッシュを無効にし、詳細ページへ戻ります。表示が変わらないときは、先行認可、再取得、フォーム同期、更新のどこで値が止まったかを確認めます。

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `server wrapper` と `client component` が `getById` を1回ずつ呼びます。2回呼ぶのはなぜですか。

A. `server wrapper` は画面を描く前に認証・認可・未発見を分けるために呼びます。`client component` は通過後のデータをキャッシュに載せます。画面から再読み込みできる状態を作るためです。両方とも同じ `getById` の認可を通るので、権限のない人に対象の実在を先に知らせません。代わりに同じデータを2回取得する通信コストがかかります。

Q2. 一般ユーザーが他人の `/user/他人のid` を開くと編集ボタンが消えるだけですか。

A. 違います。`getById` の権限チェックで `FORBIDDEN` になり、データそのものを受け取れません。「ボタンが非表示になる」のではなく「ページ自体が出ない」という作ります。

Q3. 期限の列を `format(new Date(task.dueDate), ...)` ではなく `formatDateOnly(task.dueDate)` で書くのはなぜですか。

A. `dueDate` は「日付だけ」の値として時刻を UTC の 0 時にそろえて保存してあるためです。`new Date()` で読み直して各自の時計に合わせると UTC より西の時間帯では前日にずれます。5 月 10 日締切のタスクが 5 月 9 日と表示され、期限切れの判定まで 1 日早まります。`formatDateOnly` は各自の時計へ直さず、UTC のまま日付の部分だけを取り出します。

追加課題: 別のユーザーを開いても前の編集値を残さない

動的な URL とフォーム初期化を組み合わせで確かめます。表示対象が変われば入力欄の値もその人の情報へ切り替わる必要があります。

前提は管理者でログインし、名前の違う練習用ユーザーを 2 人開けることです。Day 06 の登録操作で作成したアカウントを使えます。

1 人目の詳細から編集を開き、名前を「未保存の名前」へ変えます。保存せず、アドレス欄へ `/user` を入力し、一覧へ戻ってください。

2 人目の編集を開きます。名前とメールアドレスが 2 人目の値で、未保存の名前が残っていなければ成功です。

`src/app/user/[id]/edit/user-edit-client.tsx` のデータ取得と `reset` を読み、どのユーザーの値を渡しているか説明します。

前の名前が残る場合は取得した `user` が変わったときにフォームを初期化しているか確認してください。確認後は保存せず一覧へ戻ります。1 人目の詳細も開き、名前が元のままであることを確かめます。ロールと有効状態は変更しません。

次回予告

Day 30 ではいよいよ完成版を Vercel に公開します。30 日間コツコツ作ってきたタスク管理アプリを、世界中からアクセスできる状態にしましょう。

Day 29 完成形コード(参照用)

4 ファイルの完成形は、上の「完成コード全体」に載せた

`src/app/user/[id]/page.tsx`、`user-detail-client.tsx`、
`edit/page.tsx`、`edit/user-edit-client.tsx` です。販売用 ZIP に完成版の `src/` は入っていないため、各 Step の確認ポイントと「完成コード全体」を正本として照合してください。

次に読むもの

- 前の日: Day 28^{*1}
- 次の日: Day 30^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}

-
1. https://drive.google.com/file/d/1gnCqq_w3xfJxrKoltDW7UMydml93Jy7v/view?usp=drivesdk
 2. <https://drive.google.com/file/d/1brFAifMLznqc1jDyTDLukCIHgXL2JwXL/view?usp=drivesdk>
 3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk
 4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

- 言葉の意味: 用語集^{*6}

奥付

タイトル	Day 29: ユーザー詳細・編集ページを作ろう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。

6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>